UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
INSTITUTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

BERNARDO AFONSO NUNES
MELLANIE PEREIRA GUARILHA

ExPhish: Inteligência artificial explicável para detecção de phishing

RIO DE JANEIRO
2026

BERNARDO AFONSO NUNES
MELLANIE PEREIRA GUARILHA

ExPhish: Inteligência artificial explicável para detecção de phishing

Trabalho de conclusão de curso de graduação apresentado ao Instituto de Computação da Universidade Federal do Rio de Janeiro como parte dos requisitos para obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Prof. Gabriel Rodrigues Caldas de Aquino

RIO DE JANEIRO
2026

Nunes, Bernardo Afonso
N963e ExPhish: inteligência artificial explicável para detecção de phishing / Bernardo Afonso Nunes e Mellanie Pereira Guarilha. – Rio de Janeiro, 2026.

70 f.

Orientador: Gabriel Rodrigues Caldas de Aquino.

Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) - Universidade Federal do Rio de Janeiro, Instituto de Computação, Bacharel em Ciência da Computação, 2026.

1. Phishing. 2. DistilBERT. 3. LLMs. 4. SHAP. 5. RAG. I. Guarilha, Mellanie Pereira. II. Aquino, Gabriel Rodrigues Caldas de (Orient.). III. Universidade Federal do Rio de Janeiro, Instituto de Computação. IV. Título.

BERNARDO AFONSO NUNES
MELLANIE PEREIRA GUARILHA

ExPhish: Inteligência artificial explicável para detecção de phishing

Trabalho de conclusão de curso de graduação apresentado ao Instituto de Computação da Universidade Federal do Rio de Janeiro como parte dos requisitos para obtenção do grau de Bacharel em Ciência da Computação.

Aprovado em 06 de Agosto de 2026.

BANCA EXAMINADORA:

gov.br Documento assinado digitalmente
GABRIEL RODRIGUES CALDAS DE AQUINO
Data: 13/08/2026 08:44:48-0300
Verifique em https://validar.iti.gov.br

Prof. Gabriel Rodrigues Caldas de Aquino, D.Sc.
(Instituto de Computação - UFRJ)

gov.br Documento assinado digitalmente
SILVANA ROSSETTO
Data: 12/08/2026 20:57:49-0300
Verifique em https://validar.iti.gov.br

Profa. Silvana Rossetto, D.Sc.
(Instituto de Computação - UFRJ)

gov.br Documento assinado digitalmente
GABRIEL MARTINS DE OLIVEIRA COSTA
Data: 13/08/2026 15:20:01-0300
Verifique em https://validar.iti.gov.br

Gabriel Martins de Oliveira Costa, M.Sc.
(Doutorando PESC-COPPE - UFRJ)

À minha mãe, Elizabete, por nunca ter deixado de acreditar em mim e por sempre enxergar o meu potencial em cada etapa da minha jornada acadêmica e de vida.

Bernardo Nunes

À minha irmã, Melissa. Minha maior inspiração, minha primeira referência e, de tantas formas, a melhor parte de mim.

Mellanie Pereira

AGRADECIMENTOS

Agradeço a todos os professores e funcionários que compõem a Universidade Federal do Rio de Janeiro (UFRJ), os quais resistem ao contínuo desmantelamento da educação e da pesquisa no país e mantêm esta universidade como um centro de excelência educacional e uma força motriz para transformações científicas e sociais que tive o prazer de testemunhar nos últimos anos. Ao meu orientador, Prof. Gabriel Rodrigues Caldas de Aquino, pelas orientações, pela paciência e por ter aceitado participar deste trabalho. Agradeço também à Mellanie por sua participação no projeto e por contribuir com sua competência técnica, cujo capricho e atenção aos detalhes fizeram com que este trabalho atingisse um nível de qualidade que eu dificilmente conseguiria alcançar sozinho. À minha mãe, Elizabete, por ser minha maior referência de perseverança, resiliência e excelência acadêmica e profissional. Por fim, agradeço a todos os colegas de curso e da universidade pela colaboração e pela fraternidade ao longo desses anos.

Bernardo Nunes

Gostaria de agradecer, primeiramente, à Universidade Federal do Rio de Janeiro (UFRJ) e ao Instituto de Computação por serem a minha segunda casa e a base de todo o meu conhecimento nestes anos de formação. Ao meu orientador, Prof. Gabriel Rodrigues Caldas de Aquino, pelas valiosas orientações para que este trabalho se tornasse possível. Aos professores que fizeram parte da minha formação, por todo o conhecimento compartilhado, pela dedicação e por contribuírem, cada um à sua maneira, para a profissional que estou me tornando. Aos meus amigos Nathan, Seth, Rafael, João Paulo e Nicholas, por terem compartilhado comigo tantos momentos dessa jornada e por tornarem os dias da faculdade mais leves. À Eduarda, por todas as caronas para o Fundão, por me ouvir falar de programação mesmo sem entender nada e, principalmente, por estar sempre ao meu lado durante todos esses anos. Ao Bernardo, por dividir cada etapa desse projeto comigo e pelo convite para ser sua dupla nessa pesquisa. À minha mãe, Junne, por sempre embalar meus sonhos e me ajudar a chegar até aqui. Às minhas irmãs, Melissa e Millena, por estarem sempre ao meu lado, por serem minhas melhores amigas e por todo o apoio durante os momentos mais difíceis da graduação. À minha namorada, Maria Julia, por sempre acreditar em mim e por estar ao meu lado em cada etapa. Este trabalho também tem um pouco de você. A todos que fizeram parte dessa caminhada, meu muito obrigada.

Mellanie Pereira

RESUMO

A crescente ameaça à segurança e à privacidade de empresas e indivíduos representada pelo phishing evidencia a necessidade de soluções robustas para mitigar os riscos dessa técnica de golpe. Nesse cenário, este trabalho apresenta o desenvolvimento do ExPhish: um sistema de apoio à decisão, estruturado em arquitetura de microsserviços, para detecção e explicação de e-mails de phishing, voltado à proteção do usuário final contra e-mails maliciosos. A solução utiliza o modelo DistilBERT para a classificação de mensagens e emprega modelos de linguagem de larga escala (LLMs) para a geração de explicabilidade. A metodologia fundamenta-se no uso da técnica SHAP (SHapley Additive exPlanations) para mapear as palavras que mais contribuíram para a classificação realizada pelo DistilBERT, cujos dados alimentam o prompt do LLM. Adicionalmente, utiliza-se a técnica de Geração Aumentada por Recuperação (RAG) para enriquecer as explicações com informações extraídas de guias e manuais especializados em combate ao phishing. O desenvolvimento envolveu o ajuste fino do classificador via LoRA, a estruturação de um banco de dados vetorial e a integração dos componentes em uma arquitetura escalável, com aplicação de mecanismos de resiliência. Os resultados indicam que a combinação da precisão do DistilBERT com a interpretabilidade fornecida pelo SHAP e pelo LLM permite que o usuário compreenda os critérios de segurança violados. Conclui-se que o uso de documentos externos e métodos de atribuição de relevância aumenta a confiabilidade do sistema e promove a educação digital. Por fim, o trabalho discute limitações de latência e aponta para a evolução do mecanismo de busca semântica e da infraestrutura de implantação.

Palavras-chave: Phishing; DistilBERT; LLMs; SHAP; RAG; Explicabilidade; Microsserviços.

ABSTRACT

The growing threat to the security and privacy of companies and individuals posed by phishing highlights the need for robust solutions to mitigate the risks of this scam technique. In this scenario, this work presents the development of ExPhish: a decision support system, structured in a microservices architecture, for detecting and explaining phishing emails, aimed at protecting end-users against malicious emails. The solution utilizes the DistilBERT model for message classification and employs Large Language Models (LLMs) for explainability generation. The methodology is based on the use of the SHAP (SHapley Additive exPlanations) technique to map the words that most contributed to the classification performed by DistilBERT, providing data that feeds the LLM prompt. Additionally, Retrieval-Augmented Generation (RAG) is used to enrich the explanations with information extracted from specialized anti-phishing guides and manuals. The development involved fine-tuning the classifier via LoRA, structuring a vector database, and integrating the components into a scalable architecture with the application of resilience mechanisms. The results indicate that the combination of BERT's precision with the interpretability provided by SHAP and the LLM allows users to understand the specific security criteria violated. It is concluded that the use of external documents and relevance attribution methods increases system reliability and promotes digital education. Finally, the work discusses latency limitations and points toward the evolution of the semantic search mechanism and deployment infrastructure.

Keywords: Phishing; DistilBERT; LLM; SHAP; RAG; Explainability; Microservices.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura original do modelo Transformer 17
Figura 2 – Diagrama da arquitetura de componentes lógicos da solução 27
Figura 3 – Diagrama de atividades do fluxo de análise 31
Figura 4 – Diagrama de atividades do fluxo de *feedback* 33
Figura 5 – Tela de submissão do e-mail 38
Figura 6 – Tela de processamento 39
Figura 7 – Tela de resultados 40
Figura 8 – Tela de resultado com a explicação da IA expandidas 41
Figura 9 – Tela de resultado com a relevância dos tokens expandida 42
Figura 10 – Tela de resultado com o contexto recuperado expandida 43
Figura 11 – Tela de resultado com o formulário de feedback 44
Figura 12 – Diagrama ER 48

LISTA DE TABELAS

Tabela 1 – Métricas do treinamento com LoRA 51
Tabela 2 – Desempenho do classificador de *phishing* 58
Tabela 3 – Distribuição das avaliações das justificativas para os e-mails de phishing. 60
Tabela 4 – Quartis das avaliações das justificativas para e-mails legítimos. 61
Tabela 5 – Latência total média por faixa de tamanho da entrada 63

LISTA DE QUADROS

Quadro 1 – Requisitos Funcionais 25
Quadro 2 – Requisitos Não Funcionais 26
Quadro 3 – Descrição dos componentes lógicos da arquitetura 27
Quadro 4 – Tecnologias utilizadas 36
Quadro 5 – Estrutura da tabela `analyses` 48
Quadro 6 – Estrutura da tabela `feedback` 49
Quadro 7 – Template de prompt utilizado para a geração de explicações. 54

SUMÁRIO

1	**INTRODUÇÃO**	**13**
1.1	ORGANIZAÇÃO DO TRABALHO	15
2	**CONCEITOS BÁSICOS**	**16**
2.1	ENGENHARIA SOCIAL E PHISHING	16
2.2	MODELOS DE LINGUAGEM DE GRANDE ESCALA (LLMS)	16
2.3	AJUSTE FINO EM APRENDIZADO DE MÁQUINA	17
2.4	RETRIEVAL-AUGMENTED GENERATION (RAG)	18
2.5	INTELIGÊNCIA ARTIFICIAL EXPLICÁVEL (XAI) E APOIO À DECISÃO	19
2.6	INTERPRETABILIDADE VIA VALORES DE SHAPLEY (SHAP)	19
2.7	ARQUITETURA DE MICROSSERVIÇOS	20
2.8	MECANISMOS DE RESILIÊNCIA EM SISTEMAS DISTRIBUÍDOS	21
3	**TRABALHOS RELACIONADOS**	**22**
4	**PROPOSTA**	**24**
4.1	VISÃO GERAL	24
4.2	REQUISITOS DO SISTEMA	24
4.2.1	**Requisitos Funcionais**	**24**
4.2.2	**Requisitos Não Funcionais**	**26**
4.3	ARQUITETURA LÓGICA	27
4.3.1	**Motor de Detecção e Explicabilidade**	**28**
4.3.1.1	Fase de Detecção	28
4.3.1.2	Fase de Atribuição de Relevância	29
4.3.1.3	Fase de Geração de Justificativa	29
4.3.2	**Mecanismos de Resiliência**	**30**
4.3.3	**Degradação Graciosa**	**30**
4.4	FLUXO DE FUNCIONAMENTO	30
4.4.1	**Fluxo de Análise**	**30**
4.4.2	**Fluxo de Feedback**	**32**
5	**IMPLEMENTAÇÃO E INTEGRAÇÃO DO SISTEMA**	**35**
5.1	ARQUITETURA E ORGANIZAÇÃO DO REPOSITÓRIO	35
5.2	TECNOLOGIAS E FERRAMENTAS UTILIZADAS	35
5.3	INTERFACE DO USUÁRIO	38
5.4	ORQUESTRADOR DE REQUISIÇÕES	44
5.4.1	**Contratos de API**	**44**
5.4.1.1	Endpoints Externos (Interface → Orquestrador)	45
5.4.1.2	Endpoint interno (Orquestrador → Motor)	46

5.4.1.3 Respostas de Erro . . . 47
5.4.1.4 Mecanismos de Resiliência . . . 47
5.4.1.5 Base de Dados Operacional . . . 47
5.5 MOTOR DE DETECÇÃO E EXPLICABILIDADE . . . 49
5.5.1 Dataset e Preparação dos Dados . . . 49
5.5.2 Módulo de Classificação e Interpretabilidade Local . . . 50
5.5.2.1 Seleção do Modelo e Tokenização . . . 50
5.5.2.2 Ajuste fino do classificador usando LoRA . . . 50
5.5.2.3 Extração de Relevância com SHAP . . . 51
5.5.3 Base de Conhecimento Vetorial (RAG) . . . 51
5.5.3.1 Ingestão e Estruturação de Documentos . . . 51
5.5.3.2 Mecanismo de Busca Semântica . . . 52
5.5.4 Módulo de Síntese de Explicação . . . 52
5.5.4.1 Engenharia de Prompt . . . 53
5.5.4.2 Fluxo de Execução e Orquestração . . . 55
5.5.5 API FastAPI e Gerenciamento do Ciclo de Vida . . . 55
5.6 IMPLANTAÇÃO E INFRAESTRUTURA . . . 55
6 EXPERIMENTOS E RESULTADOS . . . 57
6.1 AVALIAÇÃO QUANTITATIVA . . . 57
6.1.1 Avaliação das Justificativas Geradas . . . 58
6.1.2 Avaliação de Latência e Desempenho Computacional . . . 62
6.1.3 Exemplo de Explicação com Recuperação de Conhecimento (RAG) . . . 63
7 CONCLUSÃO . . . 66
REFERÊNCIAS . . . 68

1 INTRODUÇÃO

Nas últimas décadas, o avanço das tecnologias de comunicação digital transformou profundamente a forma como as pessoas se relacionam, realizam transações financeiras e compartilham informações. No entanto, esse crescimento acelerado também trouxe consigo o aumento de ameaças cibernéticas, entre as quais o *phishing* se destaca como uma das mais comuns e prejudiciais. Esse tipo de ataque tem como objetivo enganar usuários, induzindo-os a revelar informações sensíveis, como senhas, dados bancários e credenciais de acesso.

O *phishing* continua sendo um dos principais métodos utilizados por criminosos para iniciar ataques cibernéticos, uma vez que sua eficiência se baseia na incapacidade da grande maioria das vítimas de identificar a veracidade de um e-mail. De fato, de acordo com dados recentes levantados por Censuswide (2026), apenas 3% dos participantes de uma pesquisa composta de 2000 pessoas conseguiram identificar se um e-mail se tratava de *phishing* ou não. Além disso, o erro humano continua a desempenhar um papel significativo, contribuindo para cerca de 60% das violações de segurança (VERIZON, 2026). Complementarmente, a CSO Online aponta que aproximadamente 80% dos incidentes de segurança estão relacionados a ataques de *phishing*, gerando perdas financeiras estimadas em US$ 17.700 por minuto. Esses números evidenciam a magnitude e a persistência desse tipo de ameaça, reforçando a necessidade de estratégias mais eficazes de prevenção e detecção.

Para mitigar esse problema, sistemas de detecção de *phishing* são amplamente usados e tradicionalmente são implementados através de filtros baseados em regras ou em algoritmos de aprendizado de máquina clássicos, como Naive Bayes e Regressão Logística. Embora essas abordagens apresentem bons resultados no geral, elas frequentemente enfrentam dificuldades para identificar padrões complexos em e-mails de *phishing* (TARAPIAH et al., 2025).

Paralelo ao contexto abordado anteriormente em cibersegurança, a popularização e disseminação em massa dos *Large Language Models* (LLMs), desde o lançamento do ChatGPT em 2022, tem impulsionado inúmeras aplicações desse tipo de modelo em diversas áreas. A capacidade desses modelos em interpretar e entender o contexto de textos, através da arquitetura de transformers baseada em atenção (VASWANI et al., 2017), possui potencial e aplicabilidade enormes no campo de *Natural Language Processing* (VOGELSANG; FISCHBACH, 2024).

Embora as *Large Language Models* (LLMs) apresentem vantagens valiosas, elas compartilham a limitação inerente aos modelos baseados em redes neurais profundas: a natureza de "caixa-preta". Isso implica que seus processos internos de processamento de dados e tomada de decisão permanecem, em grande parte, obscuros e incompreensíveis

aos seres humanos. No entanto, essa opacidade pode ser mitigada pelo uso de técnicas de Inteligência Artificial Explicável (XAI), aproveitando a própria sofisticação linguística desses modelos para gerar justificativas em linguagem natural.

Conforme explorado por Lim et al. (2025), modelos de última geração, como o Llama-3 e o GPT-4o, podem ser empregados não apenas na identificação técnica de anomalias como e-mails fraudulentos, mas na tradução de indicadores complexos em narrativas legíveis. Ao fundamentar a detecção em uma explicação compreensível, o sistema transcende a função de filtro passivo para se tornar uma ferramenta robusta de suporte à decisão. Essa abordagem permite mitigar a dependência de especialistas e fortalecer a postura de segurança do usuário final, capacitando-o a compreender criticamente os vetores de ataque aos quais está exposto.

Apesar do potencial dos LLMs como sistemas de apoio à decisão, a implementação prática enfrenta desafios críticos relacionados à confiabilidade das informações geradas. Um dos principais obstáculos é o fenômeno das *hallucinations* (alucinações), em que o modelo gera explicações plausíveis, porém factualmente incorretas ou sem fundamentação lógica. Diante disso, essas falhas podem comprometer severamente a integridade das justificativas de segurança, levando o usuário final a perder a confiança no sistema ou, em casos mais graves, a ignorar alertas legítimos devido a explicações inconsistentes.

Para mitigar essas limitações, a arquitetura de *Retrieval-Augmented Generation* (RAG) surge como uma solução robusta ao ancorar as respostas do modelo em fontes de dados externas e verificáveis. Ao integrar o processo de recuperação de documentos proposto originalmente por Lewis et al. (2021) e expandido em revisões recentes da literatura (GAO et al., 2024), o sistema deixa de depender exclusivamente do conhecimento estático de seus pesos internos para consultar repositórios de ameaças em tempo real. Essa abordagem não apenas reduz drasticamente a incidência de alucinações, mas também oferece uma flexibilidade essencial, permitindo que as referências técnicas utilizadas na explicação sejam customizadas e atualizadas de acordo com o contexto organizacional em que a ferramenta é aplicada. Assim, o RAG transforma o LLM em uma ferramenta de segurança mais adaptável, fornecendo suporte decisório fundamentado em evidências concretas e dinâmicas.

O presente trabalho tem como objetivo propor o ExPhish: um sistema de apoio à decisão baseado em modelos de linguagem (LLMs) com o uso da técnica RAG para a classificação de e-mails de *phishing*, estruturado em uma arquitetura de microsserviços. A adoção da arquitetura de microsserviços permite que os componentes do sistema sejam desenvolvidos, testados e evoluídos de forma independente, favorecendo a escalabilidade e o isolamento de falhas entre os módulos. Além da detecção, o sistema visa fornecer explicações fundamentadas sobre as ameaças identificadas, capacitando o usuário final a validar a veracidade da comunicação e tomar decisões informadas com maior segurança.

Para validação da proposta, os objetivos específicos deste trabalho são:

- **Implementar** um protótipo funcional do ExPhish, estruturado em arquitetura de microsserviços, voltado à identificação de ameaças cibernéticas em comunicações eletrônicas.
- **Implementar** um mecanismo de orquestração responsável pela coordenação entre componentes, pelo gerenciamento de requisições e pela aplicação de mecanismos de resiliência (*timeout, retry* e *circuit breaker*) e degradação graciosa, garantindo a disponibilidade do sistema, bem como um mecanismo de coleta e armazenamento de *feedback* dos usuários sobre as justificativas geradas, permitindo o refinamento futuro do modelo de detecção.
- **Avaliar** quantitativamente o desempenho do classificador de *phishing* desenvolvido, mensurando sua capacidade de distinguir corretamente mensagens legítimas de tentativas de *phishing* por meio de métricas consolidadas de classificação binária.
- **Avaliar** a eficácia das justificativas geradas pelo sistema, considerando critérios de correção, completude, fidelidade e clareza, refletindo sobre sua utilidade como ferramenta de suporte à decisão do usuário final.

1.1 ORGANIZAÇÃO DO TRABALHO

Este trabalho está organizado em sete capítulos conforme descrito a seguir:

- O **Capítulo 1** introduz o tema, delimitando o problema, os objetivos e o estado da arte na literatura;
- O **Capítulo 2** fundamenta os conceitos de *phishing*, modelos de linguagem, arquiteturas de processamento textual, arquitetura de microsserviços e outros conceitos;
- O **Capítulo 3** apresenta trabalhos relacionados na literatura;
- O **Capítulo 4** descreve a solução proposta, detalhando a arquitetura do sistema e seus componentes;
- O **Capítulo 5** apresenta a implementação técnica e a integração dos componentes do sistema;
- O **Capítulo 6** apresenta os resultados experimentais obtidos;
- O **Capítulo 7** apresenta as considerações finais.

2 CONCEITOS BÁSICOS

2.1 ENGENHARIA SOCIAL E PHISHING

A Engenharia Social representa um conjunto de técnicas de manipulação psicológica utilizadas por agentes maliciosos para induzir indivíduos a realizar ações específicas ou divulgar informações confidenciais. No contexto da segurança cibernética, o *phishing* é a vertente mais difundida dessa prática, utilizando comunicações eletrônicas — como e-mails, SMS ou mensagens instantâneas — que mimetizam entidades confiáveis para enganar o destinatário.

Diferente de ataques puramente técnicos, como a exploração de vulnerabilidades de *software*, o *phishing* direciona-se ao "elo mais fraco" da cadeia de segurança: o usuário humano. Conforme observado na literatura recente, a eficácia desses ataques reside no uso de gatilhos psicológicos, como senso de urgência, autoridade ou medo. Historicamente, essas mensagens apresentavam características detectáveis por filtros heurísticos simples, como erros ortográficos grosseiros e domínios visualmente discrepantes. Entretanto, a evolução das capacidades generativas de inteligência artificial permitiu que atacantes automatizassem a criação de conteúdos altamente personalizados e gramaticalmente impecáveis, degradando a eficácia de sistemas de detecção baseados em regras (AFANE et al., 2024).

O desafio contemporâneo não reside apenas em bloquear o conteúdo, mas em capacitar o usuário a reconhecer vetores de ataque cada vez mais sutis. Ataques de *Spear Phishing*, por exemplo, utilizam contextos específicos do alvo para aumentar a credibilidade da farsa. Nesse cenário, a detecção automática deve evoluir de uma resposta binária ("Seguro" ou "Inseguro") para um sistema de apoio à decisão. Ao fornecer explicações sobre os elementos de persuasão e as inconsistências técnicas presentes na mensagem, o sistema auxilia na construção de uma consciência situacional, transformando o usuário de uma vítima potencial em um agente ativo na verificação da legitimidade da comunicação recebida.

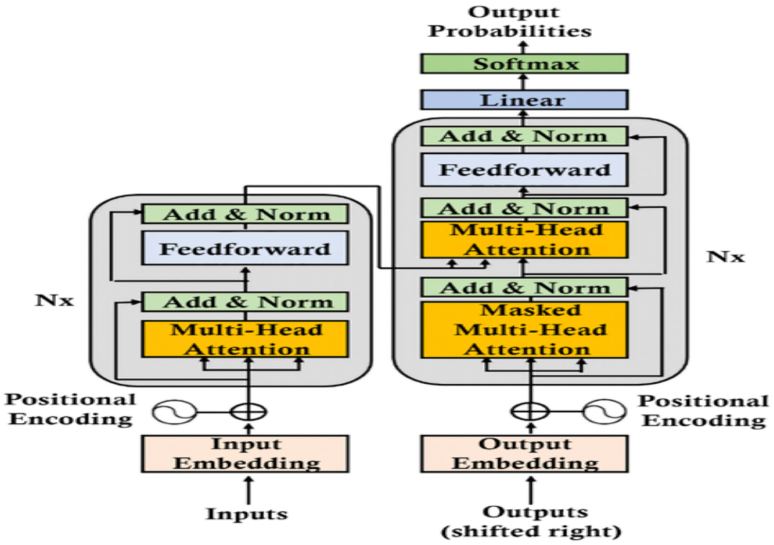
2.2 MODELOS DE LINGUAGEM DE GRANDE ESCALA (LLMS)

Os *Large Language Models* (LLMs) representam uma mudança de paradigma no processamento de linguagem natural, fundamentando-se na arquitetura *Transformer*. Diferente de modelos anteriores baseados em redes neurais recorrentes (RNNs), que processavam informações de forma sequencial e frequentemente perdiam o contexto de sentenças longas, os *Transformers* utilizam o mecanismo de *Self-Attention* (Auto-Atenção). Este mecanismo permite que o modelo atribua pesos diferenciados a cada palavra de um texto, relacionando-as simultaneamente, independentemente da distância entre elas.

No contexto da detecção de *phishing*, essa capacidade de absorção contextual é crucial. Um ataque de engenharia social muitas vezes não depende de palavras-chave maliciosas isoladas, mas sim de uma narrativa que constrói uma falsa legitimidade. Os LLMs conseguem identificar discrepâncias semânticas sutis — como um tom de urgência incompatível com o suposto remetente ou solicitações de dados que contradizem políticas institucionais conhecidas — capturando a intenção por trás da mensagem de forma que classificadores clássicos não conseguem.

Entretanto, apesar da sofisticação linguística, os LLMs operam de forma probabilística, prevendo o próximo *token* com base em padrões estatísticos aprendidos durante o treinamento. Essa característica dá origem ao fenômeno das *hallucinations* (alucinações), onde o modelo pode gerar explicações extremamente convincentes, porém factualmente errôneas ou desconectadas da realidade técnica do e-mail analisado. Conforme discutido por Kalai et al. (2025), a propensão a alucinar pode degradar a confiabilidade do sistema de apoio à decisão, tornando imperativo o uso de mecanismos de controle e ancoragem de conhecimento, como o RAG (LEWIS et al., 2021), que é uma técnica para incluir contexto ao prompt de uma LLM para evitar halucinações e melhorar a qualidade das respostas, para garantir que a explicabilidade gerada seja fidedigna aos dados reais.

Figura 1 – Arquitetura original do modelo Transformer



Fonte: Vaswani et al. (2017)

2.3 AJUSTE FINO EM APRENDIZADO DE MÁQUINA

Ajuste Fino (*fine-tuning*) (DODGE et al., 2020) consiste em uma técnica em aprendizado de máquina utilizada para adaptar um modelo previamente treinado a tarefas específicas. Esse processo é realizado por meio da continuidade do treinamento do modelo em um novo conjunto de dados, geralmente menor e mais especializado, permitindo que os

parâmetros aprendidos durante o pré-treinamento sejam ajustados para um determinado domínio ou problema.

Em modelos baseados em redes neurais profundas, especialmente aqueles fundamentados na arquitetura *Transformer*, o ajuste fino ocorre após uma etapa inicial denominada pré-treinamento (*pre-training*), na qual o modelo aprende representações gerais da linguagem a partir de grandes volumes de texto não rotulado. Durante o ajuste fino, o modelo passa por novas iterações de treinamento utilizando dados rotulados relacionados à tarefa alvo, como classificação de texto, análise de sentimentos ou detecção de e-mails de *phishing*.

O principal objetivo do ajuste fino é especializar o conhecimento previamente adquirido, permitindo que o modelo capture padrões específicos do domínio de aplicação sem a necessidade de treinamento completo desde o início. Essa abordagem reduz significativamente o custo computacional e o volume de dados necessários, além de melhorar o desempenho em tarefas específicas quando comparada a modelos treinados exclusivamente com dados limitados.

2.4 RETRIEVAL-AUGMENTED GENERATION (RAG)

O *Retrieval-Augmented Generation* (RAG) (LEWIS et al., 2021) é uma arquitetura que combina a capacidade generativa dos LLMs com sistemas de recuperação de informações em bases de dados externas. Enquanto um LLM convencional gera respostas baseando-se exclusivamente no conhecimento estático adquirido durante seu treinamento, o RAG permite que o modelo consulte fontes de dados dinâmicas e verificáveis em tempo real antes de formular uma saída. Esta abordagem é fundamental para mitigar as alucinações, uma vez que a geração do texto passa a ser ancorada (*grounded*) em evidências recuperadas, e não apenas em previsões probabilísticas de *tokens*.

O funcionamento do RAG ocorre tipicamente em três etapas principais:

1. **Recuperação (Retrieval):** A partir de uma entrada do usuário (ex: um e-mail suspeito), o sistema transforma o texto em vetores numéricos (*embeddings*) e busca em uma base de dados vetorial os documentos ou fragmentos técnicos mais semanticamente semelhantes.

2. **Ampliação (Augmentation):** O contexto recuperado é inserido no *prompt* enviado ao LLM, fornecendo ao modelo informações atualizadas sobre táticas de *phishing*, domínios maliciosos conhecidos ou indicadores de comprometimento (IoCs).

3. **Geração (Generation):** O LLM processa o *prompt* enriquecido e gera uma explicação em linguagem natural que referencia diretamente os fatos recuperados, aumentando a precisão e a auditabilidade da resposta.

A principal vantagem do RAG em sistemas de apoio à decisão é a flexibilidade contextual. Diferente do processo de ajuste fino, que exige o retreinamento oneroso do modelo para cada nova atualização, o RAG permite que a base de conhecimento seja constantemente alimentada com novos exemplos de ataques. Para o usuário final, isso se traduz em explicações mais seguras e personalizadas, onde o sistema pode citar normas específicas da organização ou ameaças recém-descobertas, garantindo que o suporte à decisão seja fundamentado em dados concretos e contemporâneos.

2.5 INTELIGÊNCIA ARTIFICIAL EXPLICÁVEL (XAI) E APOIO À DECISÃO

A Inteligência Artificial Explicável (*Explainable AI* - XAI) (GUNNING et al., 2019) surge como uma resposta à crescente complexidade dos modelos de aprendizado profundo, que operam como "caixas-pretas" de difícil interpretação humana. O objetivo da XAI não é apenas otimizar a precisão da detecção, mas fornecer transparência aos processos de decisão da máquina. No contexto da segurança cibernética, a explicabilidade é o que transforma um alerta de segurança em um sistema de apoio à decisão, capacitando o usuário a discernir entre comunicações legítimas e tentativas de fraude.

Diferente das abordagens clássicas de XAI, que utilizam mapas de calor ou pesos numéricos de difícil compreensão para leigos, os LLMs permitem uma explicabilidade linguística e textual. Através de técnicas como o *Chain-of-Thought* (CoT) (WEI et al., 2022), o modelo é induzido a decompor seu raciocínio em etapas lógicas antes de emitir o veredito final. Essa decomposição permite que o usuário acompanhe o "pensamento" do sistema, identificando quais elementos do e-mail — como o tom de voz, a estrutura do cabeçalho ou a inconsistência do *link* — foram determinantes para a classificação de risco.

Conforme discutido por Wang et al. (2025), a eficácia de um sistema de apoio à decisão em *phishing* é medida pela sua capacidade de reduzir a incerteza do usuário. Ao utilizar o RAG para ancorar essas explicações em fatos e evidências, o sistema promove a literacia digital: o usuário não apenas evita o ataque imediato, mas aprende a reconhecer os padrões de ameaça para interações futuras. Portanto, a integração entre LLM, RAG e XAI cria uma camada de defesa proativa, onde a inteligência artificial atua como um assistente analítico que amplia a capacidade crítica do indivíduo diante de ameaças digitais.

2.6 INTERPRETABILIDADE VIA VALORES DE SHAPLEY (SHAP)

No âmbito da XAI, uma das abordagens mais robustas para quantificar a contribuição de cada característica (*feature*) em uma predição é o SHAP (*SHapley Additive exPlanations*). Baseado na teoria dos jogos cooperativos, o SHAP atribui a cada atributo um valor de importância que representa o quanto aquela variável alterou a previsão final em relação à previsão média do modelo.

A força do SHAP reside na sua fundamentação matemática, que garante propriedades como a consistência e a completude. O valor de Shapley para uma característica i é calculado pela média ponderada de todas as possíveis combinações (coalizões) de atributos, conforme a fórmula:

$$\phi_i(f, x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} [f(S \cup \{i\}) - f(S)] \tag{2.1}$$

Onde:

- M é o número total de características;
- S é um subconjunto de características que não contém i;
- $f(S)$ é a saída do modelo treinada apenas com as características do conjunto S;
- $[f(S \cup \{i\}) - f(S)]$ representa a contribuição marginal da característica i ao ser adicionada à coalizão.

Apesar de sua precisão matemática, a aplicação do SHAP em modelos de detecção de *phishing* tradicionais apresenta um desafio de comunicação com o usuário final. Enquanto o SHAP indica que o "Atributo X contribuiu com +0.15 para o risco", um sistema de apoio à decisão baseado em LLM consegue traduzir essa importância em uma justificativa semântica, como: "O remetente utiliza um domínio que mimetiza uma instituição bancária, prática comum em ataques de engenharia social". Portanto, este trabalho propõe que a explicabilidade linguística atue como uma ponte de comunicação, tornando os conceitos abstratos da interpretabilidade técnica acessíveis ao usuário leigo.

2.7 ARQUITETURA DE MICROSSERVIÇOS

De acordo com Lewis e Fowler (2014), a arquitetura de microsserviços é uma abordagem para desenvolvimento de *software* na qual uma aplicação é estruturada como um conjunto de serviços pequenos e independentes, cada um executando em seu próprio processo e comunicando-se através de mecanismos leves, geralmente APIs REST. Cada microsserviço implementa uma funcionalidade de negócio específica e opera de forma autônoma, permitindo implantação independente e uso de diferentes linguagens de programação e tecnologias de armazenamento de dados.

No contexto do presente trabalho, a arquitetura de microsserviços foi adotada para permitir separação clara de responsabilidades entre os módulos da solução. Os componentes funcionais operam como serviços independentes que se comunicam via interfaces bem definidas, permitindo desenvolvimento e evolução autônoma de cada módulo. Esta modularidade facilita a manutenção, possibilita escalabilidade e permite que cada componente utilize as tecnologias mais adequadas às suas responsabilidades específicas.

2.8 MECANISMOS DE RESILIÊNCIA EM SISTEMAS DISTRIBUÍDOS

Em sistemas distribuídos baseados em microsserviços, falhas são inevitáveis e devem ser tratadas como parte normal da operação (NEWMAN, 2021). Diferentemente de arquiteturas monolíticas, onde um único ponto de falha pode comprometer todo o sistema, microsserviços aprimoram a resiliência ao isolar problemas em serviços individuais, evitando que a indisponibilidade de um componente comprometa toda a aplicação. Para lidar com essas falhas de forma controlada e prevenir propagação em cascata, adotam-se padrões de estabilidade que aumentam a resiliência do sistema. Os mecanismos fundamentais **retry, timeout e circuit breaker**, atuam de forma complementar para garantir degradação controlada da funcionalidade em cenários de falha (NEWMAN, 2021):

- ***Retry***: O mecanismo de retry retenta automaticamente operações que falharam, sendo útil para lidar com falhas temporárias como picos de carga, perda de pacotes ou indisponibilidade momentânea de serviços. Ao permitir novas tentativas, evita-se que problemas transitórios escalem desnecessariamente, como abertura de *circuit breakers* ou erros visíveis ao usuário final, quando uma simples retentativa resolveria o problema.

- ***Timeout***: Para que *retries* sejam eficazes, é fundamental estabelecer um tempo máximo de espera por resposta. *Timeout* é o mecanismo que define este limite temporal de uma operação. Em sistemas distribuídos, a ausência de *timeout* pode resultar em *threads* ou processos bloqueados indefinidamente aguardando respostas que nunca chegarão, levando à exaustão de recursos e indisponibilidade do sistema.

- ***Circuit breaker***: Quando falhas persistem mesmo após *retries*, o *circuit breaker* atua como última linha de defesa. Ele é um mecanismo de proteção que monitora falhas em chamadas a serviços externos e, após um limiar de falhas consecutivas, "abre o circuito", fazendo requisições subsequentes falharem imediatamente sem tentar contatar o serviço problemático. Esta estratégia protege tanto o cliente, evitando esperas desnecessárias, quanto o serviço em falha, evitando sobrecarga adicional.

Estes mecanismos atuam de forma complementar: *retries* lidam com falhas transitórias, *timeouts* previnem bloqueios, e *circuit breakers* protegem contra indisponibilidades persistentes. A aplicação consistente destes padrões é fundamental para garantir resiliência em arquiteturas distribuídas.

3 TRABALHOS RELACIONADOS

Vários trabalhos recentes investigaram o potencial das Large Language Models (LLMs) no domínio de explicabilidade (XAI) e segurança cibernética, especificamente na detecção e explicação de ataques de *phishing* (LIM et al., 2025).

O trabalho de Lim et al. (2025) descreve o EXPLICATE, um framework de detecção de *phishing* explicável que integra um classificador baseado em aprendizado de máquina, técnicas de explicabilidade (LIME e SHAP) e um modelo de linguagem para traduzir justificativas técnicas em linguagem natural. Os autores demonstram que é possível manter elevada precisão na detecção de ataques (98,4%) ao mesmo tempo em que fornecem explicações compreensíveis aos usuários, contribuindo para aumentar a transparência e a confiança no sistema.

Em Martens et al. (2026), a ideia central é propor o XAIstories, uma abordagem que utiliza modelos de linguagem de grande porte (LLMs) para transformar explicações técnicas geradas por métodos de Inteligência Artificial Explicável (XAI), como SHAP e explicações contrafactuais, em narrativas textuais mais compreensíveis para usuários.

Já em Zeng e Zhu (2026), é aprofundado como os valores SHAP podem ser utilizados em conjunto com modelos de linguagem de grande porte (LLMs) para tornar explicações de modelos de aprendizado de máquina mais acessíveis a usuários não técnicos. Mais especificamente, os autores propõem o uso de um LLM pré-treinado para traduzir as explicações numéricas geradas pelo SHAP em descrições em linguagem natural, preservando a fidelidade das informações enquanto aumentam sua clareza e compreensão. Os resultados indicam que essa abordagem melhora significativamente a interpretabilidade das predições de modelos complexos, permitindo que usuários compreendam com maior facilidade os fatores que influenciaram as decisões do sistema.

Nessa mesma linha, o estudo conduzido por Wang et al. (2025) foca na percepção humana e na utilidade das explicações geradas por IA em contextos de segurança cibernética. Wang et al. (2025) demonstram que a confiança do usuário é influenciada pela clareza da narrativa fornecida pelo modelo durante a triagem de mensagens suspeitas. O trabalho sugere que a economia de esforço cognitivo ocorre quando a IA pré processa o contexto do e-mail e destaca as intenções maliciosas de forma legível. Dessa forma, a explicabilidade se torna um componente essencial para que o usuário final compreenda os vetores de ataque e tome decisões informadas, fortalecendo a segurança de forma colaborativa entre humano e máquina.

Complementarmente, Zhang et al. (2025) exploram o desempenho técnico de LLMs na classificação de ameaças em comparação com métodos tradicionais. Zhang et al. (2025) argumentam que, embora as LLMs sejam frequentemente vistas como caixas pretas, a aplicação de técnicas de inteligência artificial explicável permite que o modelo exponha

seu raciocínio lógico.

Por fim, o trabalho de Guttikonda et al. (2025) propõe um framework baseado em Retrieval-Augmented Generation (RAG) para aumentar a transparência de modelos de Machine Learning e Deep Learning. A abordagem combina Large Language Models (LLMs) com bases de conhecimento específicas de domínio para fornecer explicações claras e interativas sobre as predições dos modelos, tornando-as mais acessíveis a usuários não técnicos. Os autores demonstram a aplicabilidade da solução em diferentes áreas, como saúde, finanças e manufatura, evidenciando seu potencial como uma abordagem escalável para promover interpretabilidade, confiança e compreensão em sistemas de inteligência artificial.

Enquanto investigações pregressas, a exemplo de Lim et al. (2025) e Martens et al. (2026), evidenciam a eficácia de traduzir artefatos matemáticos de atribuição de relevância — como o SHAP — em narrativas compreensíveis em linguagem natural, trabalhos como os de Zeng e Zhu (2026) e Wang et al. (2025) sublinham o impacto dessa transparência na redução do esforço cognitivo e na elevação da confiança do usuário final durante a tomada de decisão. Além disso, a análise comparativa frente aos métodos tradicionais conduzida por Zhang et al. (2025) e a estratégia de ancoragem contextual com RAG proposta por Guttikonda et al. (2025) respaldaram a escolha de integrar uma base de conhecimento externa ao pipeline analítico. Dessa forma, o conjunto de trabalhos relacionados serviu de subsídio para desenhar uma arquitetura em microsserviços capaz de mitigar alucinações de LLMs e entregar explicações fundamentadas e transparentes para a mitigação de ataques de phishing.

4 PROPOSTA

Conforme apresentado no Capítulo 1, ataques de *phishing* representam uma ameaça crescente à segurança digital, explorando vulnerabilidades humanas por meio de engenharia social. Sistemas automatizados de detecção existem, porém frequentemente operam como "caixa-preta", fornecendo classificações sem justificativas compreensíveis. Esta limitação dificulta que usuários avaliem a veracidade das informações e tomem decisões. Diante deste cenário, o presente trabalho propõe o ExPhish, um sistema de apoio à decisão que não apenas detecta tentativas de *phishing*, mas também fornece explicações fundamentadas sobre as ameaças identificadas. Este capítulo apresenta a arquitetura proposta, descrevendo seus requisitos, componentes e estratégias de integração.

4.1 VISÃO GERAL

O ExPhish consiste em um sistema de apoio à decisão para detecção e explicação de *phishing*, estruturado em arquitetura de microsserviços, que integra processamento de linguagem natural (NLP) e inteligência artificial explicável (XAI) para auxiliar o usuário final na tomada de decisão. A arquitetura é composta por cinco componentes lógicos, sendo três componentes funcionais que se comunicam via APIs REST: Interface do Usuário (UI), Orquestrador de Requisições (OR) e Motor de Detecção e Explicabilidade (MDE) e dois componentes de persistência: Base de Dados Operacional e Base de Conhecimento.

A arquitetura modular proposta permite que cada componente seja evoluído independentemente, facilitando manutenção e escalabilidade. O orquestrador incorpora mecanismos de resiliência para garantir disponibilidade mesmo em cenários de falha parcial de componentes individuais. O detalhamento da arquitetura, fluxos de funcionamento e técnicas empregadas é apresentado nas seções subsequentes deste capítulo.

4.2 REQUISITOS DO SISTEMA

Requisitos de software são especificações das funcionalidades que um sistema deve oferecer e das restrições sob as quais deve operar (VALENTE, 2020). Conforme classifica Valente (2020), requisitos dividem-se em duas categorias principais: **requisitos funcionais**, que descrevem o que o sistema faz, ou seja, suas funcionalidades, e **requisitos não funcionais**, que especificam restrições que o sistema deve atender, como desempenho, disponibilidade e escalabilidade.

4.2.1 Requisitos Funcionais

O Quadro 1 apresenta os requisitos funcionais do sistema, ou seja, suas funcionalidades.

Quadro 1 – Requisitos Funcionais

RF	Descrição
01	O sistema deve permitir ao usuário submeter o conteúdo de um e-mail para análise.
02	O sistema deve validar que o conteúdo submetido não está vazio e não excede 2.500 caracteres antes de encaminhá-lo para processamento, exibindo mensagem de erro caso o critério não seja atendido.
03	O sistema deve retornar a classificação do e-mail como *phishing* ou legítimo.
04	O sistema deve retornar uma pontuação de confiança (0–100) associada à classificação.
05	O sistema deve gerar uma explicação em linguagem natural que justifique a classificação.
06	O sistema deve identificar e destacar os *tokens*/palavras mais relevantes para a classificação com suas respectivas contribuições (SHAP).
07	O sistema deve fundamentar as explicações em fontes externas de conhecimento sobre *phishing*, recuperando trechos relevantes com base no conteúdo do e-mail analisado (RAG).
08	O sistema deve apresentar ao usuário os resultados da análise, incluindo veredito de classificação, pontuação de confiança, explicação em linguagem natural, contribuições dos *tokens* mais relevantes para a decisão (SHAP) e contexto de conhecimento recuperado (RAG).
09	O sistema deve permitir ao usuário avaliar a análise fornecida, informando se a classificação estava correta, se a explicação foi útil e, opcionalmente, adicionando *tags* predefinidas e um comentário em texto livre.
10	O sistema deve armazenar cada análise concluída, incluindo o conteúdo do e-mail analisado, o resultado da classificação, a pontuação de confiança e a explicação gerada.
11	O sistema deve armazenar o *feedback* do usuário vinculado à análise correspondente, preservando a rastreabilidade entre avaliação e resultado.
12	O sistema deve exibir mensagens de erro claras quando a análise falhar, independentemente da causa da falha.
13	O sistema deve exibir um indicativo visual de processamento contínuo enquanto a análise estiver em execução.

Os requisitos RF01 a RF08 correspondem ao fluxo principal de submissão e análise, enquanto os requisitos RF09 a RF11 viabilizam a coleta de *feedback* para refinamento futuro do modelo de detecção, conforme descrito nos objetivos específicos deste trabalho. Para garantir uma boa experiência e transparência, o RF12 estabelece a obrigação de comunicar falhas de forma clara em qualquer etapa, e o RF13 garante a exibição de indicativos visuais de progresso, mitigando a percepção de demora durante o tempo de processamento. Segundo Zensec SOC employees (2026), um e-mail de *phishing* tem, em média, 1.058 caracteres. Assim, o limite de 2.500 caracteres definido no RF02 atende com folga à maioria dos ataques reais e previne gargalos de latência no pipeline, aspecto que será aprofundado na Seção 6.1.2.

4.2.2 Requisitos Não Funcionais

O Quadro 2 apresenta os requisitos não funcionais do sistema, ou seja, as restrições que os requisitos funcionais devem respeitar.

Quadro 2 – Requisitos Não Funcionais

RNF	Descrição
01	O sistema deve aplicar políticas de resiliência para garantir disponibilidade mesmo com falhas parciais nos serviços.
02	O sistema deve entregar a resposta ao usuário normalmente mesmo que o armazenamento da análise falhe, registrando o problema internamente sem interromper o processamento.
03	A latência máxima de uma análise completa não deve ultrapassar 120 segundos, abrangendo todas as etapas do processamento, classificação, identificação e geração de explicação.
04	O sistema deve ser modular, permitindo que seus componentes sejam desenvolvidos, testados e implantados independentemente.
05	O sistema deve ser implantável em ambientes containerizados, facilitando sua distribuição e execução em diferentes infraestruturas.
06	O sistema deve registrar os eventos relevantes de cada etapa do processamento, viabilizando auditoria e depuração de falhas.
07	O sistema deve continuar operando normalmente mesmo que o serviço de recuperação de contexto esteja indisponível, omitindo o contexto da resposta sem interromper a análise.
08	O sistema deve proteger dados sensíveis em repouso, evitando vazamentos por meio de mecanismos de proteção adequados.

De forma complementar, os requisitos não funcionais estabelecem as restrições de arquitetura, resiliência e desempenho do sistema. Os requisitos RNF04 e RNF05 garantem a modularidade e a conteinerização da aplicação, enquanto RNF01, RNF02 e RNF07 asseguram a tolerância a falhas por meio de políticas de resiliência e degradação graciosa. A transparência e a rastreabilidade são definidas pelo requisito RNF06, enquanto o requisito RNF08 é responsável pela segurança dos dados.

Dentre essas restrições operacionais, o limite de 120 segundos (RNF03) foi definido para equilibrar o alto custo computacional dos LLMs com a experiência do usuário. Segundo as heurísticas de Nielsen (1993), esperas acima de 10 segundos exigem indicadores visuais de progresso para reter a atenção do usuário, necessidade atendida pelo requisito RF13. Vale ressaltar que o foco do trabalho é validar a integração do pipeline analítico (DistilBERT, SHAP, RAG e LLMs), e não otimizar o tempo de resposta da aplicação em si. Por isso, a arquitetura manteve o modelo síncrono, deixando a adoção de mensageria assíncrona ou filas de processamento, que mitigariam o impacto dessa latência, como oportunidade para trabalhos futuros. O comportamento real do sistema diante desse limite temporal é analisado na Seção 6.1.2.

4.3 ARQUITETURA LÓGICA

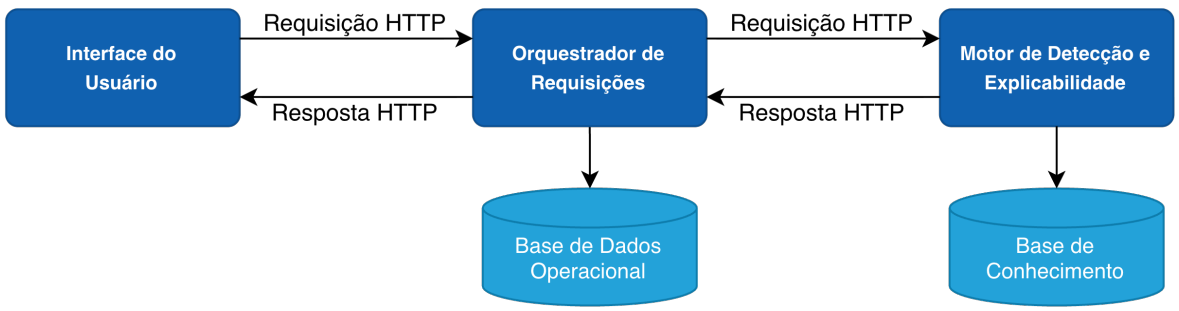
A arquitetura lógica da solução foca na organização dos requisitos funcionais do sistema, definindo como o domínio do problema é decomposto em componentes abstratos e como as responsabilidades são distribuídas entre eles (KRUCHTEN, 1995). Para orientar essa divisão de forma modular, o projeto adota o estilo arquitetural de microsserviços, com separação clara de responsabilidades entre os módulos, promovendo o desacoplamento.

O sistema é estruturado em cinco componentes lógicos (CL) principais, detalhados no Quadro 3 e ilustrados na Figura 2. Essa abordagem garante que esses elementos possam evoluir e operar de maneira independente, comunicando-se estritamente por contratos bem definidos.

Quadro 3 – Descrição dos componentes lógicos da arquitetura

CL	**Nome**	**Descrição**
01	Interface do Usuário (UI)	Responsável por capturar o conteúdo de e-mails submetidos pelo usuário (RF01), validar a entrada (RF02), exibir os resultados da análise (RF08) e coletar *feedback* (RF09). Comunica-se exclusivamente com o Orquestrador de Requisições via API REST.
02	Orquestrador de Requisições (OR)	Atua como coordenador central: recebe requisições da Interface, encaminha-as ao Motor, aplica políticas de resiliência (RNF01) e persiste dados operacionais na base relacional (RF10, RF11).
03	Motor de Detecção e Explicabilidade (MDE)	Executa o pipeline completo de inteligência artificial em três fases: Detecção (RF03, RF04), Atribuição de Relevância por SHAP (RF06) e Geração de Justificativa via RAG (RF05, RF07). Recupera contexto da Base de Conhecimento quando disponível (RNF08).
04	Base de Dados Operacional	Armazena dados operacionais do sistema, incluindo análises concluídas e *feedbacks* fornecidos pelos usuários. Gerenciada exclusivamente pelo Orquestrador (RF10, RF11).
05	Base de Conhecimento	Base vetorial que indexa guias e manuais *anti-phishing* utilizados no mecanismo de Recuperação Aumentada por Geração (RAG). Acessada exclusivamente pelo Motor (RF07).

Figura 2 – Diagrama da arquitetura de componentes lógicos da solução



4.3.1 Motor de Detecção e Explicabilidade

O Motor de Detecção e Explicabilidade (CL03) é o componente responsável pela execução do processamento analítico do sistema. Sua principal função é receber o conteúdo do e-mail encaminhado pelo Orquestrador de Requisições e produzir um conjunto de informações que apoiem o usuário na tomada de decisão sobre a legitimidade da mensagem analisada.

Para atender aos requisitos funcionais do sistema, o Motor organiza seu processamento em três fases sequenciais: Detecção, Atribuição de Relevância e Geração de Justificativa. Essa decomposição permite separar responsabilidades distintas do processo analítico, favorecendo a modularidade e possibilitando a evolução independente de cada etapa.

A primeira fase é responsável por determinar a classificação do e-mail e estimar o nível de confiança associado ao resultado obtido. Em seguida, a segunda fase identifica os elementos da mensagem que exerceram maior influência sobre a decisão produzida. Por fim, a terceira fase transforma essas evidências em uma explicação textual compreensível ao usuário final, complementando-a, quando disponível, com conhecimento especializado recuperado de fontes externas.

O resultado consolidado produzido pelo Motor é composto pelo veredito de classificação, pela pontuação de confiança associada, pelas evidências de relevância identificadas durante a análise e pela justificativa em linguagem natural. Essas informações são retornadas ao Orquestrador, que se encarrega de sua persistência e posterior apresentação ao usuário.

4.3.1.1 Fase de Detecção

A Fase de Detecção é responsável por analisar o conteúdo do e-mail submetido e produzir uma decisão inicial sobre sua legitimidade. O objetivo dessa etapa é determinar se a mensagem apresenta características compatíveis com uma tentativa de *phishing* ou se corresponde a uma comunicação legítima.

Como resultado do processamento, o sistema produz um veredito de classificação e uma medida de confiança associada à decisão. Essa medida representa o grau de convicção do modelo em relação ao resultado obtido, servindo como informação complementar para o usuário final e para as etapas subsequentes do pipeline.

Por constituir a primeira etapa do processo analítico, esta fase fornece a base para a geração das explicações produzidas pelo sistema. O resultado obtido é encaminhado à etapa de atribuição de relevância, que será responsável por identificar quais elementos do e-mail exerceram maior influência sobre a classificação realizada.

4.3.1.2 Fase de Atribuição de Relevância

Com o objetivo de tornar a decisão do classificador mais transparente, a arquitetura incorpora uma etapa de atribuição de relevância responsável por identificar quais elementos do e-mail exerceram maior influência sobre o resultado produzido na fase de detecção.

Nessa etapa, cada elemento textual recebe uma medida de contribuição associada ao seu impacto na classificação realizada pelo sistema. Dessa forma, torna-se possível evidenciar quais partes da mensagem favoreceram a identificação de um possível ataque de *phishing* e quais contribuíram para caracterizá-la como uma comunicação legítima.

O resultado desse processo consiste em um conjunto de evidências interpretáveis que descrevem o comportamento do modelo para uma instância específica. Essas evidências não apenas aumentam a transparência da solução, mas também servem como insumo para a etapa subsequente de geração de justificativas.

Ao disponibilizar os principais fatores considerados durante a classificação, essa fase contribui para reduzir a percepção de caixa-preta frequentemente associada a sistemas baseados em aprendizado de máquina, permitindo que a decisão seja apresentada de forma mais compreensível e auditável ao usuário final.

4.3.1.3 Fase de Geração de Justificativa

Após a identificação das evidências mais relevantes para a classificação, o sistema executa a fase de geração de justificativa, cujo objetivo é transformar os resultados produzidos nas etapas anteriores em uma explicação compreensível ao usuário final.

Além das evidências obtidas a partir da análise do e-mail, a arquitetura prevê a utilização de uma Base de Conhecimento especializada em *phishing* e engenharia social. Essa base tem como finalidade complementar a interpretação do resultado, relacionando os indícios observados na mensagem a conceitos, práticas e padrões reconhecidos na literatura e em materiais de referência da área.

A justificativa produzida nessa etapa busca não apenas informar o resultado da classificação, mas também apresentar os motivos que levaram o sistema a chegar àquela conclusão. Dessa forma, o usuário recebe uma explicação contextualizada, capaz de relacionar os elementos identificados no e-mail aos comportamentos normalmente observados em campanhas de *phishing*.

O resultado final consiste em uma explicação textual fundamentada nas evidências produzidas pelo pipeline analítico e complementada por conhecimento especializado, contribuindo tanto para a compreensão quanto para o auxílio do usário final na tomada de decisão.

4.3.2 Mecanismos de Resiliência

O Orquestrador (CL02) implementa as seguintes estratégias para garantir disponibilidade na comunicação com o Motor de Detecção e Explicabilidade (RNF01, RNF03):

1. ***Timeout*** (RNF01, RNF03): Todas as chamadas ao Motor possuem limite de tempo configurável. Se excedido, a requisição é cancelada e um erro é retornado ao usuário.

2. ***Retry* com *Backoff* Exponencial** (RNF01): Em caso de falha temporária, o Orquestrador realiza tentativas de reenvio com esperas crescentes entre cada tentativa.

3. ***Circuit Breaker*** (RNF01): Se o Motor falha em alta proporção de requisições em curto período, o circuito abre e bloqueia novas requisições por tempo determinado, prevenindo cascata de falhas.

4.3.3 Degradação Graciosa

Para serviços auxiliares, a arquitetura adota o princípio de degradação graciosa: falhas em componentes não essenciais não interrompem o fluxo principal, apenas reduzem a completude da resposta (RNF02, RNF07).

1. **Persistência não bloqueante** (RNF02): Falhas no armazenamento da análise no Banco de Dados Operacional são registradas internamente sem interromper a resposta ao usuário.

2. **RAG opcional** (RNF07): o sistema opera normalmente caso a Base de Conhecimento esteja indisponível, omitindo o contexto da explicação gerada sem interromper o processamento.

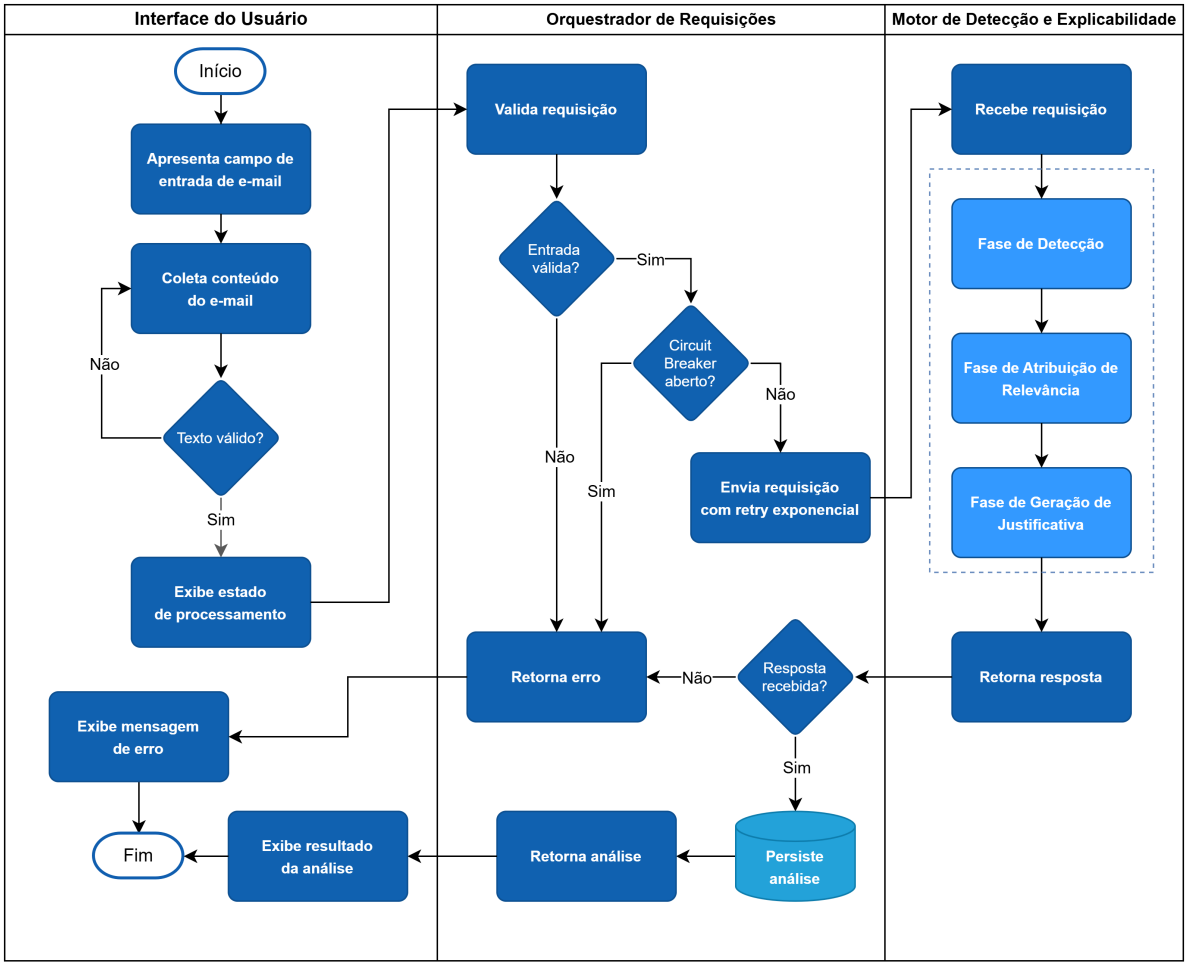
4.4 FLUXO DE FUNCIONAMENTO

Os dois fluxos principais do sistema são descritos a seguir. Cada fluxo é apresentado segundo a estrutura descritiva de casos de uso proposta por Valente (2020). Dessa forma, cada operação é apresentada a partir de um fluxo principal (também chamado de fluxo feliz), que descreve a sequência de eventos bem-sucedida em que o objetivo é alcançado, seguido pelas extensões (fluxos alternativos ou de exceção), que especificam os desvios, tratamentos de erro e comportamentos do sistema diante de falhas.

4.4.1 Fluxo de Análise

O fluxo de análise, ilustrado na Figura 3, atende aos RF01 a RF08 e RF12.

Figura 3 – Diagrama de atividades do fluxo de análise



Fluxo principal:

1. O usuário fornece o conteúdo do e-mail (RF01) e a Interface do Usuário (CL01) realiza uma validação local, verificando se o texto não está vazio e não excede 2.500 caracteres (RF02).

2. Estando a entrada válida, CL01 encaminha a requisição ao Orquestrador de Requisições (CL02) via API REST e passa a exibir a tela de processamento com o indicativo visual contínuo (RF13).

3. O Orquestrador valida a requisição no servidor e a repassa ao Motor de Detecção e Explicabilidade (CL03) aplicando uma política de *retry* exponencial.

4. O Motor executa o pipeline em três fases sequenciais (RF03 a RF07) e retorna o resultado.

5. Obtido o resultado, o Orquestrador persiste a análise na Base de Dados Operacional (CL04) de forma assíncrona e não bloqueante (RF10, RNF02).

6. O Orquestrador repassa o resultado à Interface, que apresenta ao usuário o veredito, a pontuação de confiança, a explicação, os *tokens* SHAP e o contexto RAG, encerrando o fluxo (RF08).

Fluxos de exceção:

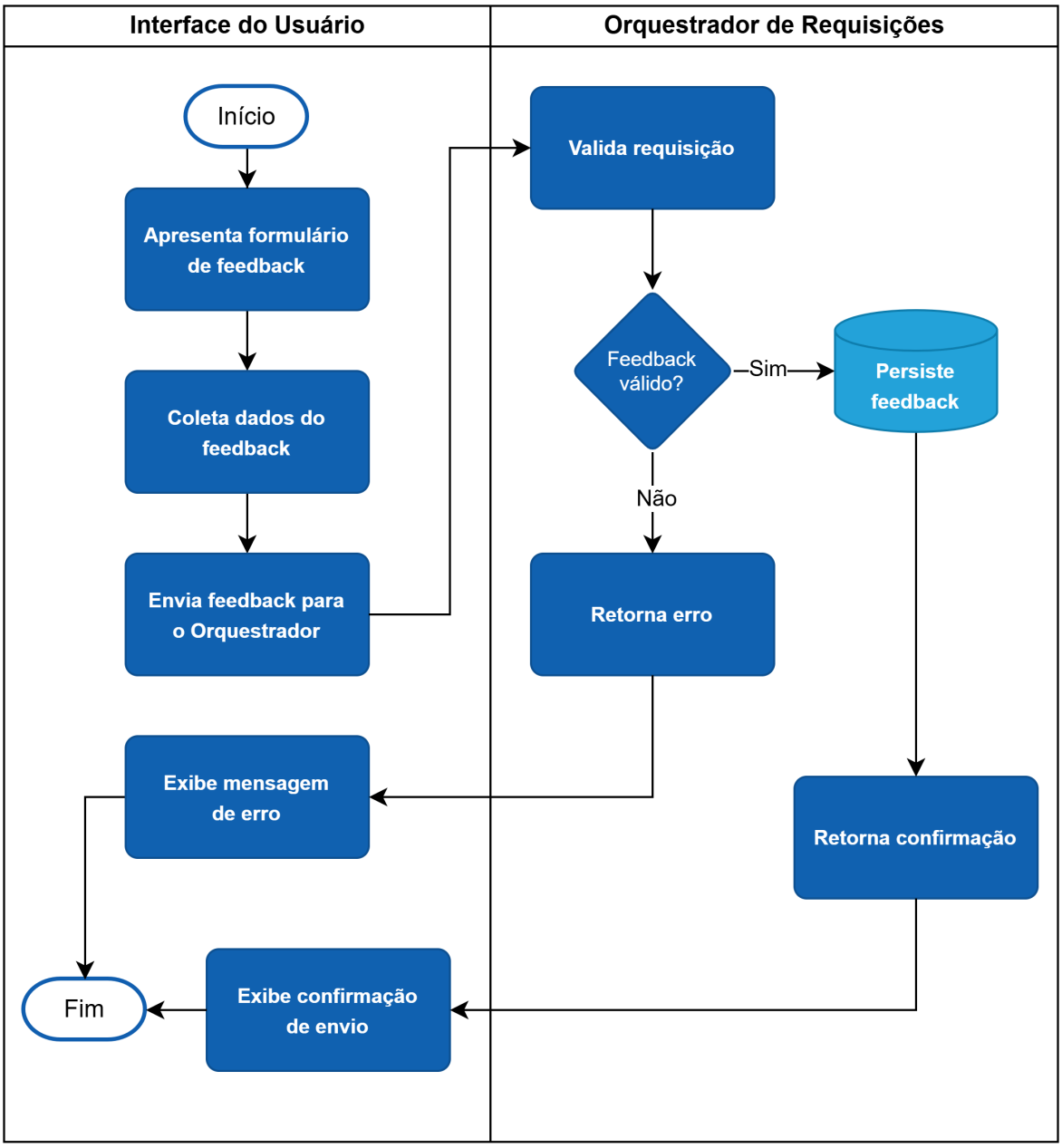
Qualquer condição de falha encerra o fluxo com a exibição de uma mensagem de erro ao usuário (RF12), sem possibilidade de retomada. São condições de encerramento antecipado:

- **Entrada inválida:** Detectada pela Interface (CL01) antes do envio ao servidor (RF02).
- **Requisição rejeitada:** Recusada pelo Orquestrador (CL02) após validação no servidor.
- **Circuit Breaker aberto:** O Orquestrador (CL02) identifica instabilidade recente no Motor e retorna erro imediato de serviço indisponível (RNF01).
- **Timeout/Falha de comunicação:** Ausência de resposta do Motor após o esgotamento das tentativas de *retry* do Orquestrador (RNF01).

4.4.2 Fluxo de Feedback

O fluxo de *feedback*, ilustrado na Figura 4, atende aos requisitos RF09, RF10 e RF12.

Figura 4 – Diagrama de atividades do fluxo de *feedback*



Fluxo principal:

1. Após visualizar o resultado da análise, o usuário preenche o formulário de *feedback*, informando se a classificação estava correta, se a explicação foi útil e, opcionalmente, selecionando *tags* predefinidas e adicionando um comentário em texto livre (RF09).

2. A Interface do Usuário (CL01) recebe as informações e encaminha os dados ao Orquestrador de Requisições (CL02) via API REST.

3. O Orquestrador valida as informações e as persiste na Base de Dados Operacional (CL04), vinculando o *feedback* ao identificador da análise correspondente (RF11).

4. O Orquestrador retorna uma mensagem de confirmação de sucesso à Interface, que a exibe ao usuário, encerrando o fluxo.

Fluxo de exceção:

Assim como no fluxo de análise, qualquer falha resulta no encerramento da operação com a respectiva sinalização ao usuário.

- **Dados inválidos ou incompletos:** Caso o Orquestrador (CL02) identifique campos obrigatórios ausentes ou em formato inválido, a requisição é rejeitada e um erro é retornado à Interface (CL01), que exibe a mensagem correspondente ao usuário e encerra o fluxo (RF12).

5 IMPLEMENTAÇÃO E INTEGRAÇÃO DO SISTEMA

Este capítulo descreve as decisões técnicas e os detalhes de implementação técnica dos componentes que compõem o ExPhish, mostrando como a arquitetura lógica proposta foi materializada no sistema.

5.1 ARQUITETURA E ORGANIZAÇÃO DO REPOSITÓRIO

Para a organização e o gerenciamento do código-fonte, adotou-se a estratégia de Monorepo (repositório único), consolidando os três componentes em um único repositório. Essa decisão arquitetural foi tomada para simplificar a gestão unificada de dependências, simplificar o versionamento conjunto, facilitar o rastreamento de mudanças que afetam múltiplos serviços simultaneamente e padronizar o processo de compilação e implantação por meio de uma única especificação do *Docker Compose*.

5.2 TECNOLOGIAS E FERRAMENTAS UTILIZADAS

Para materializar a arquitetura de microsserviços proposta e atender aos requisitos de desempenho e escalabilidade, adotou-se um conjunto diversificado de tecnologias modernas. A definição dessa pilha tecnológica foi guiada pelo princípio da heterogeneidade tecnológica. Conforme defende Newman (2021), uma das principais vantagens dos microsserviços é viabilizar a adoção de múltiplas tecnologias de forma isolada, permitindo que cada serviço utilize a linguagem, o *framework* ou o banco de dados que melhor resolva o seu domínio de problema específico, mitigando os riscos de acoplamento global do sistema e evitando os engessamentos típicos de um sistema monolítico.

Tendo em vista essa premissa teórica, o Quadro 4 sintetiza as tecnologias utilizadas para cada camada da aplicação.

Quadro 4 – Tecnologias utilizadas

Camada	Tecnologia	Versão
UI – Framework	Next.js (App Router)	15.x
UI – Biblioteca	React	19.x
UI – Componentes	shadcn/ui (Radix UI + Tailwind CSS)	—
OR – Linguagem	Go	1.22
OR – Framework	Echo	v4
0R – Injeção de Dependência	Uber Fx	—
OR – Logging	Uber Zap	—
OR – Banco de Dados	PostgreSQL	16-alpine
MDE – Linguagem	Python	3.11
MDE – Framework	FastAPI	—
MDE – Gerenciador de Dependências	uv (Astral)	—
MDE – Modelo Base	DistilBERT (`distilbert-base-uncased`)	Hugging Face
MDE – Fine-tuning	PEFT (LoRA)	—
MDE – Explainable AI	SHAP	—
MDE – Embeddings (RAG)	`all-MiniLM-L6-v2` (Sentence Transformers)	—
MDE – Banco de Dados	ChromaDB	PersistentClient
MDE – LLM	ChatOpenAI via LangChain	—
Containerização	Docker + Docker Compose	—

Interface do Usuário (UI): A concepção inicial da interface visual apoiou-se na ferramenta v0 da Vercel[1], que utiliza inteligência artificial generativa para acelerar a prototipação de telas a partir de descrições em texto. A aplicação final foi estruturada utilizando React 19.x[2] e o *framework* Next.js 15.x[3] (sob a abordagem *App Router*). Para os componentes visuais, adotou-se a biblioteca *shadcn/ui*, baseada nas primitivas do Radix UI[4] e no Tailwind CSS[5]. Embora a IA generativa tenha acelerado o rascunho inicial, em especial a etapa de prototipação do sistema, a interface passou por diversos refinamentos manuais para se adequar melhor ao fluxo geral do sistema desenvolvido.

Orquestrador de Requisições (OR): O microsserviço de orquestração foi desenvolvido utilizando a linguagem Go[6] (versão 1.22), o roteamento e o tratamento das requisições HTTP foram implementados com o framework web Echo v4[7]. Também foi utilizado o pa-

[1] https://v0.app/
[2] https://react.dev/
[3] https://nextjs.org/
[4] https://www.radix-ui.com/
[5] https://tailwindcss.com/
[6] https://go.dev/
[7] https://echo.labstack.com/

cote Uber Fx[8] para injeção de dependências e o Uber Zap[9] para a geração estruturada de logs. A linguagem Go foi escolhida para esse microsserviço devido à sua eficiência no tratamento de concorrência e baixo consumo de recursos em rede, características essenciais para um componente que atua como ponto central de roteamento. Embora a concorrência não tenha sido explorada de forma ampla nesse trabalho, a solução foi construída pensando na sua evolução futura e escalabilidade.

Motor de Detecção e Explicabilidade (MDE): O microsserviço de detecção foi desenvolvido em Python 3.11, utilizando o framework FastAPI[10] para exposição dos serviços via API REST. O ambiente virtual e o gerenciamento de dependências Python foram orquestrados com a ferramenta uv. A linguagem Python foi adotada no Motor em razão da maturidade do seu ecossistema para o aprendizado de máquina. O serviço faz uso intenso de bibliotecas avançadas de Inteligência Artificial: emprega-se o ecossistema Hugging Face[11] (Transformers e PEFT) para operar o modelo base DistilBERT e as matrizes de adaptação LoRA, a biblioteca SHAP para extrair a importância das características locais do texto, e o *framework* LangChain para orquestrar o fluxo do RAG (utilizando o modelo de *embeddings* `all-MiniLM-L6-v2`) e formular os *prompts* enviados ao LLM.

Camada de Persistência: Em vez de concentrar todos os dados em um único local, a arquitetura adotou o padrão de persistência poliglota. Isso permite que cada tipo de dado seja armazenado na tecnologia mais adequada às suas características de acesso e estrutura, em vez de forçar toda a informação do sistema em um único modelo de banco de dados (SADALAGE; FOWLER, 2019). Dados relacionais e transacionais, como análises dos e-mails e avaliações dos usuários, são armazenados pelo Orquestrador no PostgreSQL[12] (versão 16-alpine) para garantir integridade. Paralelamente, o Motor opera seu próprio banco de dados vetorial, o ChromaDB[13] (*PersistentClient*), que é otimizado exclusivamente para a indexação e busca semântica fundamentais ao mecanismo RAG.

Infraestrutura e Implantação: Por fim, o ambiente de execução foi padronizado utilizando Docker. A conteinerização garante que cada microsserviço, junto com seus respectivos bancos de dados e dependências, funcione de forma isolada e previsível, mitigando falhas de compatibilidade. Para coordenar a inicialização, a injeção de variáveis de ambiente e a rede virtual entre os contêineres, utilizou-se o *Docker Compose*[14].

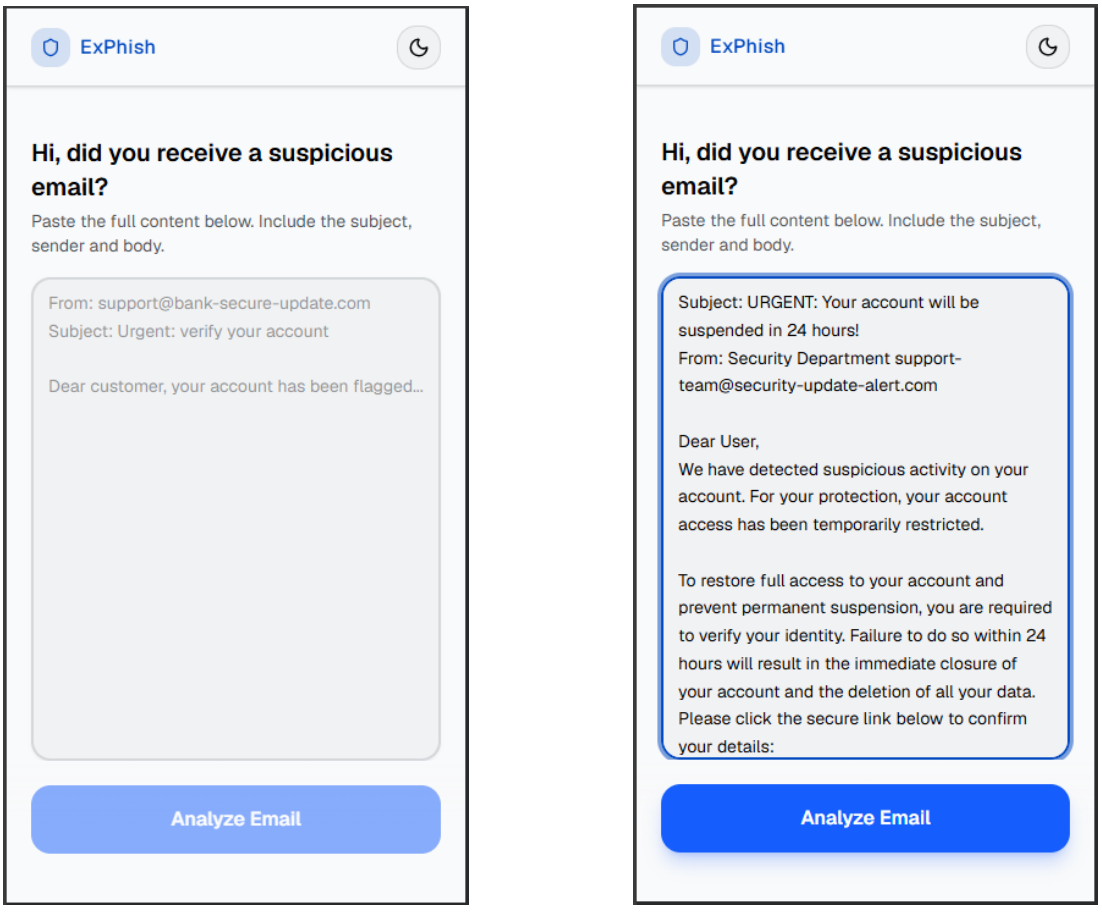
[8] https://github.com/uber-go/fx
[9] https://github.com/uber-go/zap
[10] https://fastapi.tiangolo.com/
[11] https://huggingface.co/
[12] https://www.postgresql.org/
[13] https://www.trychroma.com/
[14] https://docs.docker.com/compose/

5.3 INTERFACE DO USUÁRIO

A interface do usuário conta com 3 telas principais que acompanham o usuário desde a submissão do e-mail até a visualização completa do resultado da análise e *feedback*, incluindo a etapa intermediária de processamento.

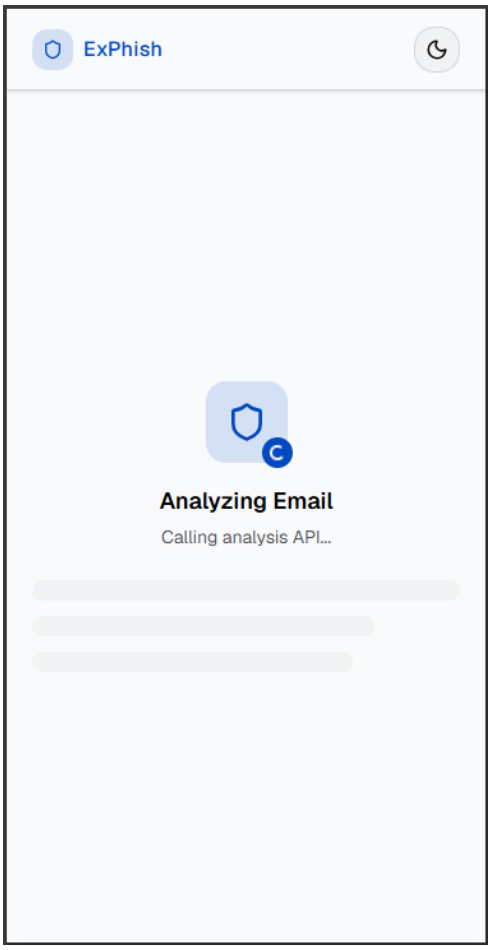
A primeira tela (Figura 5) apresenta o formulário de submissão, no qual o usuário deve colar o conteúdo do e-mail suspeito em uma área de texto, sendo orientado a incluir assunto, remetente e corpo da mensagem para que a análise disponha do máximo de contexto possível. Para garantir a integridade da análise e evitar processamento desnecessário no servidor, o botão de submissão "*Analyze Email*" permanece desabilitado enquanto o texto inserido estiver vazio ou exceder 2.500 caracteres, conforme o requisito RF02.

Figura 5 – Tela de submissão do e-mail



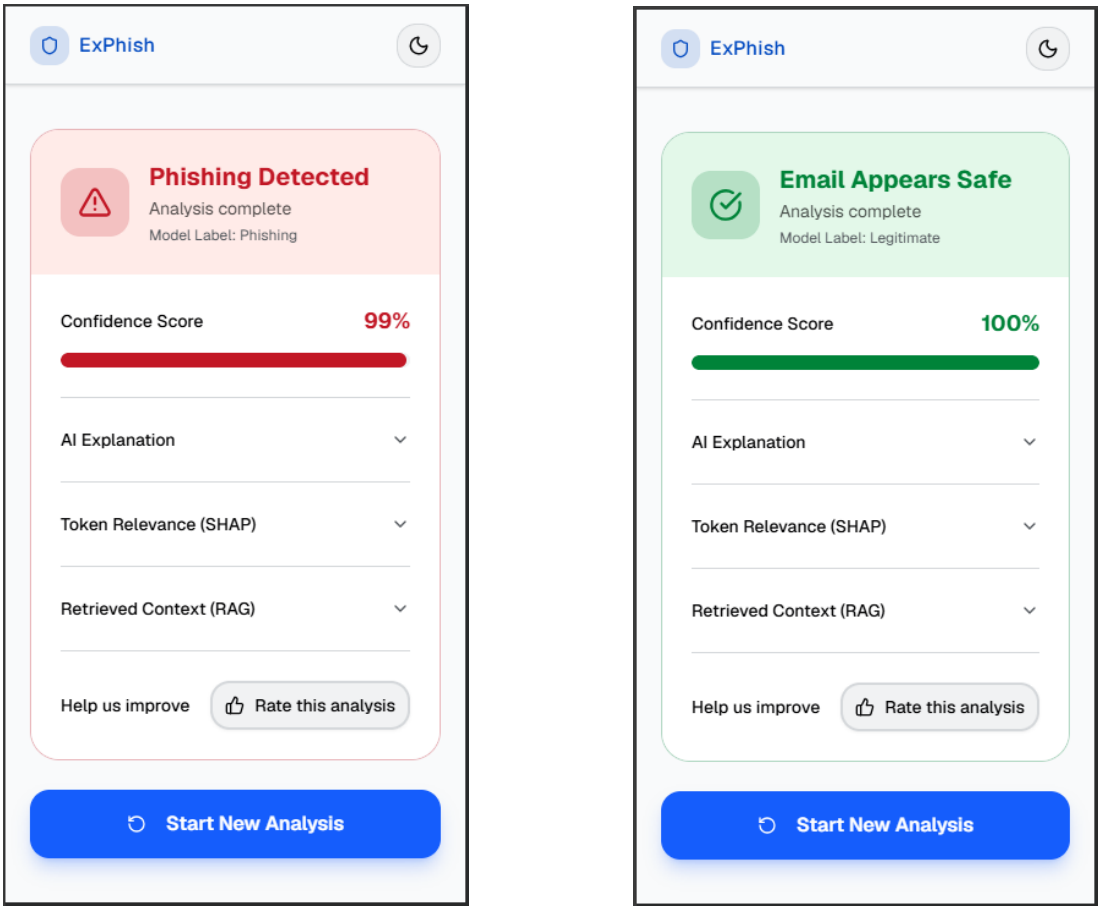
Uma vez submetida a requisição, a interface transita suavemente para a tela de processamento (Figura 6). Durante esta etapa, são exibidos indicativos visuais dinâmicos para demonstrar que o Motor está operando. Essa tela de transição permanece ativa ao longo da execução completa do *pipeline* de três fases, encerrando-se assim que o Orquestrador devolve a resposta da API (limitando-se ao tempo de latência máxima especificado no requisito RNF03).

Figura 6 – Tela de processamento



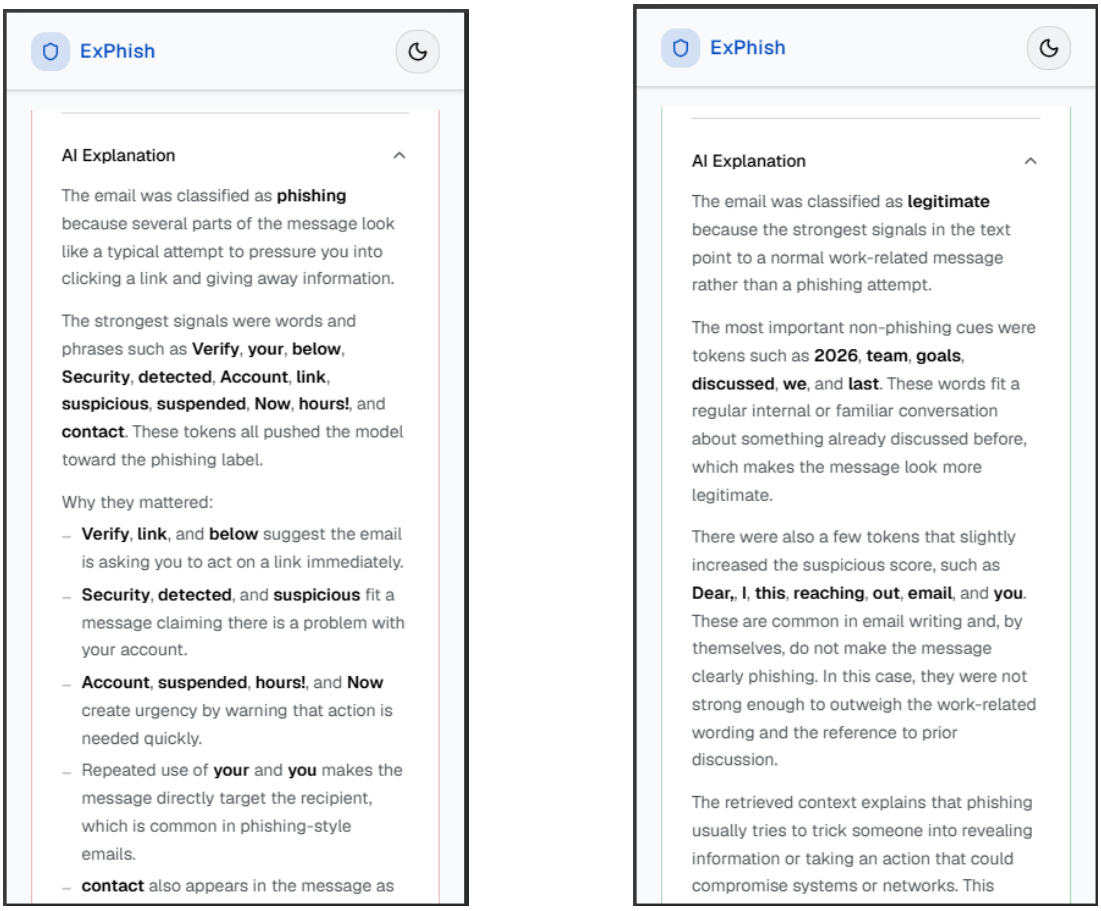
Finalizada a análise, a aplicação exibe a tela de resultados (Figura 7) com o veredito final classificando a mensagem como legítima ou *phishing*, acompanhado da barra de confiança e do percentual correspondente. Abaixo do veredito, são exibidas três seções colapsáveis que detalham os motivos da classificação.

Figura 7 – Tela de resultados



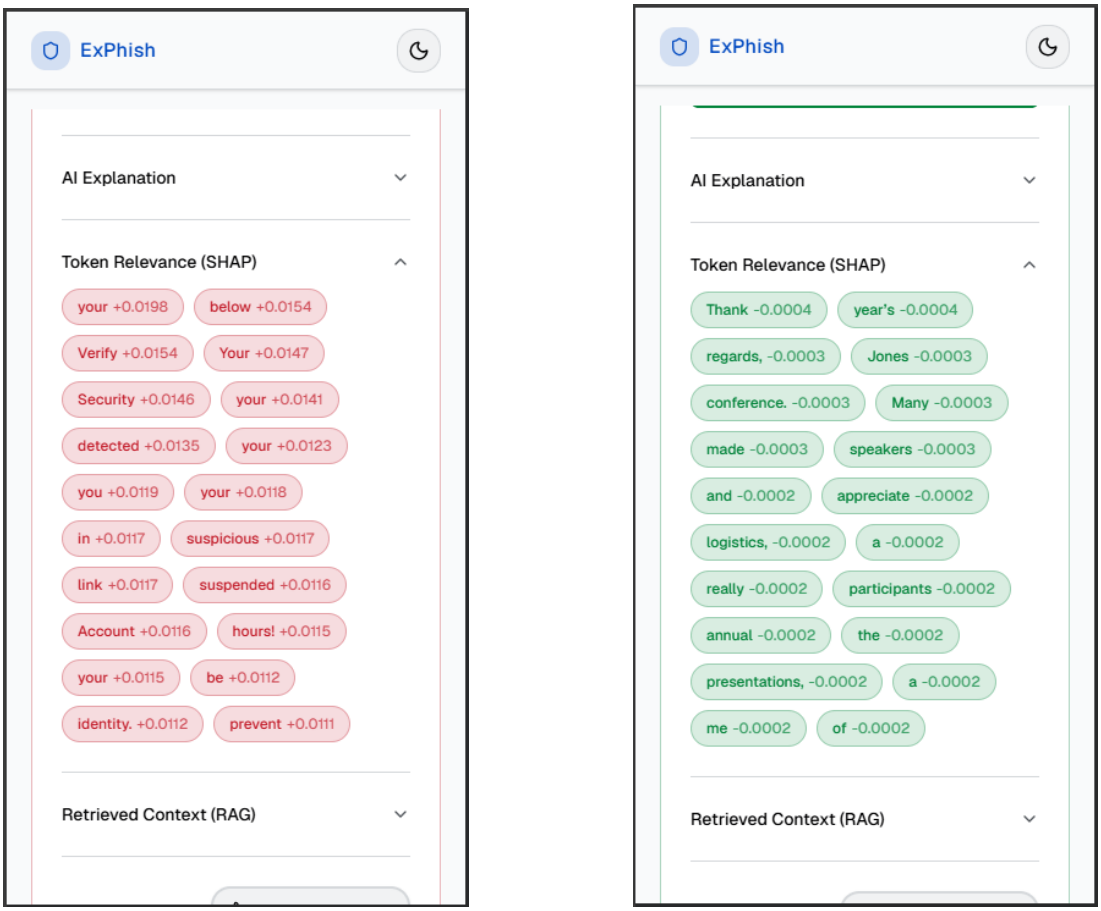
Ao expandir a primeira seção (Figura 8), o usuário tem acesso à justificativa gerada pelo LLM em linguagem natural, destacando em negrito os termos do e-mail mais relevantes para a classificação identificados a partir da atribuição de relevância calculada pelo SHAP.

Figura 8 – Tela de resultado com a explicação da IA expandidas



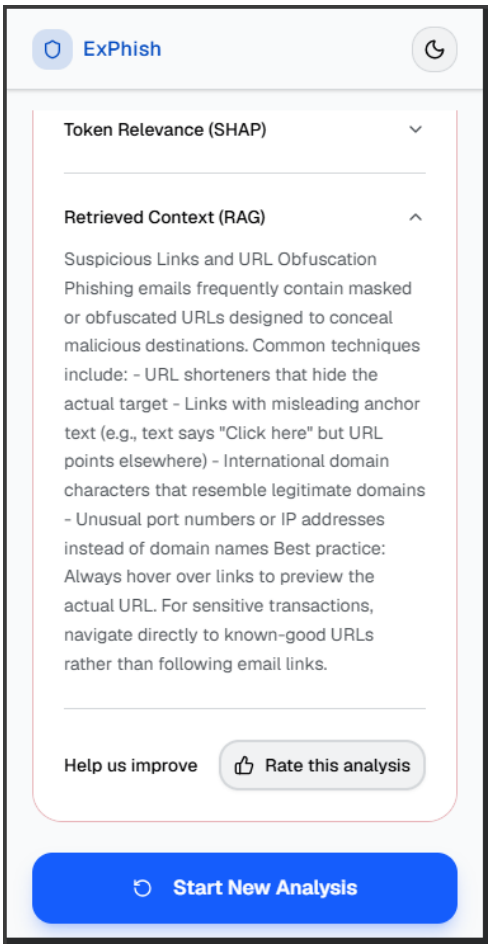
Em seguida, a segunda seção (Figura 9) apresenta o impacto direto dos *tokens* segundo a técnica SHAP. Através de um código de cores intuitivo, as palavras que mais contribuíram para a classificação como *phishing* são destacadas em fundo vermelho (indicando valores preditivos positivos), enquanto aquelas que apontam para a legitimidade são destacadas em verde (indicando valores negativos).

Figura 9 – Tela de resultado com a relevância dos tokens expandida



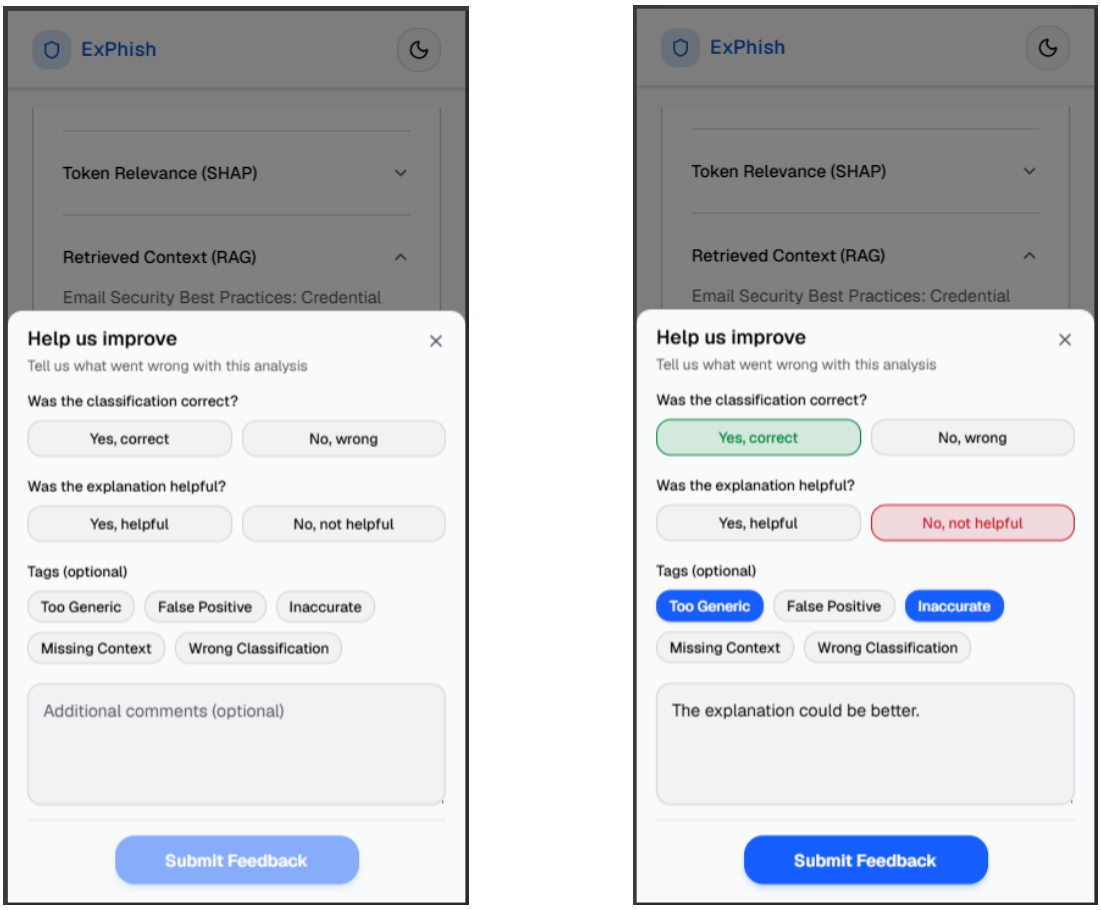
Na sequência, caso o mecanismo RAG tenha localizado documentos de apoio relevantes na base vetorial do sistema, a última seção expansível (Figura 10) exibe os trechos de documentos e/ou manuais técnicos de segurança que fundamentaram a resposta gerada pelo LLM.

Figura 10 – Tela de resultado com o contexto recuperado expandida



Encerrando o fluxo de interação, a plataforma disponibiliza um formulário de *feedback* (Figura 11), acionado por meio do botão "*Rate this analysis*" na tela de resultados (Figura 7), que eleva um painel inferior deslizante (*Bottom Sheet*). O objetivo desta interface é coletar informações para possibilitar o refinamento futuro do modelo de detecção. Para garantir a qualidade metodológica dos dados recolhidos, a submissão exige o preenchimento obrigatório de dois indicativos binários: a assertividade da classificação e a utilidade da explicação gerada. Adicionalmente, o formulário permite que o usuário selecione marcadores rápidos de contexto (*tags* predefinidas, como "Too Generic", "False Positive", "Inaccurate", "Missing Context" ou "Wrong Classification") e registre observações em um campo de texto livre antes de concluir a avaliação.

Figura 11 – Tela de resultado com o formulário de feedback



Após a submissão do *feedback*, ou a qualquer momento na tela de resultados, a interface permite que o ciclo de avaliação seja reiniciado por meio da ação de nova análise (botão "*Start New Analysis*"). O acionamento deste botão limpa o estado atual da aplicação na memória do navegador e redireciona o fluxo do usuário de volta à tela inicial de entrada de dados, viabilizando a submissão de um novo e-mail para processamento de forma contínua e sem a necessidade de recarregamento da página.

5.4 ORQUESTRADOR DE REQUISIÇÕES

O Orquestrador de Requisições atua como o principal ponto de entrada para as interações originadas na Interface do Usuário. Sua responsabilidade central é gerenciar o roteamento, validar o tráfego, aplicar as políticas de resiliência e garantir a persistência transacional dos dados, desacoplando a interface web das complexidades do Motor.

5.4.1 Contratos de API

A comunicação entre os componentes da arquitetura é realizada exclusivamente via API REST com *payloads* no formato JSON. Três *endpoints* compõem o contrato de integração do sistema sendo dois externos, consumidos pela Interface do Usuário, e um

interno, consumido pelo Orquestrador ao delegar o processamento analítico ao Motor de Detecção e Explicabilidade.

Embora a comunicação entre os microsserviços tenha sido implementada de forma síncrona, reconhece-se que este não é o modelo arquitetural ideal para lidar com a alta latência inerente às operações de LLMs. A decisão por essa abordagem foi dada para validar o funcionamento do ExPhish sem adicionar a complexidade de gerenciar ferramentas de mensageria ou filas de processamento, mantendo o foco inicial na validação do pipeline de inteligência artificial.

5.4.1.1 Endpoints Externos (Interface → Orquestrador)

Endpoint: `POST /phishing/decision`
Descrição: Ponto de entrada do sistema. Recebe o conteúdo do e-mail, orquestra a análise completa acionando o Motor, persiste o resultado no banco e retorna a decisão consolidada ao cliente (RF01–RF08).

Parâmetros da Requisição (Body):

- `email` (*string*, obrigatório): Conteúdo completo do e-mail a ser analisado (não vazio, máximo de 2.500 caracteres).

Parâmetros da Resposta (200 OK):

- `analysisId` (*UUID*): Identificador único da análise persistida.
- `predictionLabel` (*string*): Classificação binária (`"phishing"` ou `"legitimate"`).
- `predictionConfidence` (*float*): Pontuação de confiança no intervalo [0, 1].
- `explanation` (*string*): Justificativa gerada pelo LLM.
- `shapValues` (*array*): Lista de objetos com a influência de cada termo na decisão, contendo `token` (*string*), `shapValue` (*float*) e `attribution` (*string*).
- `ragContext` (*string*): Contexto recuperado da base vetorial.

Endpoint: `POST /phishing/feedback`
Descrição: Registra o *feedback* do usuário sobre a classificação e a explicação geradas para uma análise prévia (RF09, RF11).

Parâmetros da Requisição (Body):

- `analysisId` (*UUID*, obrigatório): ID da análise retornada pelo *endpoint* de decisão.
- `classificationCorrect` (*boolean*, obrigatório): Indica se o usuário concorda com a classificação.

- `explanationHelpful` (*boolean*, obrigatório): Indica se a explicação foi útil.
- `tags` (*array*, opcional): Rótulos adicionais para categorizar o *feedback*.
- `comment` (*string*, opcional): Comentário livre com observações do usuário.

Parâmetros da Resposta (200 OK):

- `id` (*integer*): Identificador numérico único do *feedback* gerado pelo banco de dados.
- `analysisId` (*UUID*): ID da análise à qual o *feedback* foi vinculado.
- `classificationCorrect` (*boolean*): Confirmação registrada sobre a classificação.
- `explanationHelpful` (*boolean*): Confirmação registrada sobre a utilidade da explicação.
- `tags` (*array*): Lista de rótulos persistidos.
- `comment` (*string* | *null*): Comentário livre persistido (`null` se não fornecido).
- `createdAt` (*string/date-time*): Data e hora (em formato ISO 8601) em que o registro foi criado.

5.4.1.2 Endpoint interno (Orquestrador → Motor)

Endpoint: `POST /v1/phishing/analysis`
Descrição: *Endpoint* interno exposto pelo Motor e consumido exclusivamente pelo Orquestrador.

Parâmetros da Requisição (Body):

- `email` (*string*, obrigatório): Conteúdo do e-mail validado pelo Orquestrador.

Parâmetros da Resposta (200 OK):

- `id` (*UUID*): Identificador de processamento interno do Motor.
- `predictionLabel` (*string*): Classificação binária (`"phishing"` ou `"legitimate"`).
- `predictionConfidence` (*float*): Pontuação de confiança no intervalo [0, 1].
- `explanation` (*string*): Justificativa gerada pelo LLM.
- `shapValues` (*array*): Lista de objetos com a influência de cada termo na decisão, contendo `token` (*string*), `shapValue` (*float*) e `attribution` (*string*).
- `ragContext` (*string*): Contexto recuperado da base vetorial.

5.4.1.3 Respostas de Erro

Todos os *endpoints* retornam erros estruturados no formato JSON, contendo um campo `error` ou `detail` descrevendo a causa da falha (RF12). Os códigos de *status* HTTP padronizados são:

`400 Bad Request:` *Payload* JSON inválido ou malformado.

`422 Unprocessable Entity:` Erro de validação dos campos (ex.: *string* muito curta ou ausência de campo obrigatório).

`500 Internal Server Error:` Erro interno do servidor ou falha crítica no processamento.

`503 Service Unavailable:` *Endpoint* do Motor ainda inicializando os modelos ou *circuit breaker* aberto no Orquestrador.

5.4.1.4 Mecanismos de Resiliência

Como o Orquestrador depende diretamente da disponibilidade do Motor para finalizar a análise, a comunicação HTTP interna entre esses dois contêineres foi protegida de forma programática.

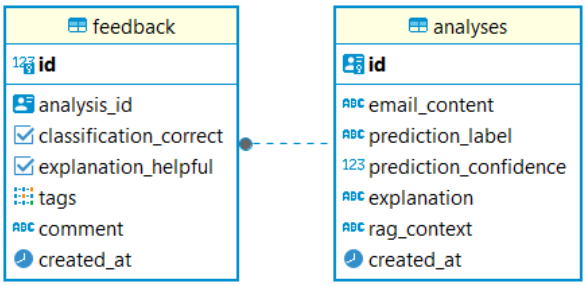
Para lidar com instabilidades transitórias da rede virtual interna, o cliente HTTP do Orquestrador foi configurado com um padrão de *Retry* com *Backoff* Exponencial. Se a conexão falhar ou expirar pelo *Timeout* de 120 segundos (imposto para satisfazer o requisito RNF03), o Orquestrador tentará reenviar a requisição com espaçamentos de tempo progressivos. Para cenários de falha severa (como a indisponibilidade total do Motor), implementou-se um *Circuit Breaker* nativo. Ao detectar uma sucessão predefinida de falhas (estado *Open*), o Orquestrador bloqueia as requisições ativas instantaneamente, retornando um erro de "serviço indisponível" para a interface sem sobrecarregar o Motor.

5.4.1.5 Base de Dados Operacional

Os registros operacionais e as métricas de avaliação do sistema são armazenados de forma estruturada no banco de dados relacional PostgreSQL. A modelagem foi estruturada por meio de duas tabelas principais interligadas, visando garantir a integridade referencial das informações geradas pelo Motor de Detecção e Explicabilidade e as respectivas avaliações submetidas pelos usuários na Interface.

A Figura 12 ilustra o Diagrama Entidade-Relacionamento do banco de dados operacional, evidenciando o vínculo entre as análises realizadas e o *feedback* coletado.

Figura 12 – Diagrama ER



O Quadro 5 detalha a estrutura da tabela `analyses`, responsável por arquivar o resultado individual de cada inferência. Pensando em viabilizar a retroalimentação e o retreinamento futuro do modelo de classificação, o conteúdo original do e-mail submetido (`email_content`) é armazenado em texto plano. Como o escopo central deste trabalho é validar a viabilidade da arquitetura, a implementação de mecanismos avançados de criptografia para a segurança dos dados em repouso não foi realizada, ainda que tenha sido mapeada pelo requisito RNF08.

Quadro 5 – Estrutura da tabela `analyses`

Campo	**Tipo**	**Descrição**
`id`	UUID	Identificador único da análise (chave primária).
`email_content`	TEXT	Conteúdo original do e-mail submetido.
`prediction_label`	VARCHAR	Rótulo da classificação inferida (ex:*phishing* ou *legitimate*).
`prediction_confidence`	FLOAT	Pontuação probabilística de confiança do modelo (entre 0 e 1).
`explanation`	TEXT	Texto contendo a justificativa final gerada pelo LLM.
`rag_context`	TEXT	Trechos dos documentos técnicos recuperados pela base vetorial.

O Quadro 6 apresenta a estrutura da tabela `feedback`, que foi modelada para extrair duas métricas centrais de avaliação do sistema: uma métrica quantitativa, que mensura se o modelo acertou ou não a predição (`classification_correct`), e uma métrica qualitativa, que avalia se a justificativa gerada foi útil (`explanation_helpful`). Essa estruturação foi fundamental para o projeto, pois forneceu exatamente os insumos práticos utilizados para a validação da ferramenta na etapa de Experimentos e Resultados (Capítulo 6). Para assegurar a higienização do banco de dados, o relacionamento de chave estrangeira com a tabela de análises (`analysis_id`) foi configurado com a restrição `ON DELETE CASCADE`, garantindo que a exclusão de uma análise resulte na deleção automática de suas avaliações, prevenindo registros órfãos.

Quadro 6 – Estrutura da tabela `feedback`

Campo	Tipo	Descrição
`id`	INTEGER	Identificador único sequencial da avaliação (chave primária).
`analysis_id`	UUID	Referência à análise avaliada (chave estrangeira).
`classification_correct`	BOOLEAN	Indicador binário sobre a precisão do veredito gerado pelo modelo.
`explanation_helpful`	BOOLEAN	Indicador binário sobre a utilidade e clareza da explicação do LLM.
`tags`	ARRAY	Lista de marcadores pré-definidos de contexto selecionados pelo usuário.
`comment`	TEXT	Observações adicionais inseridas em formato de texto livre.

Para lidar com a evolução da estrutura do banco de dados ao longo do desenvolvimento, adotou-se o uso de *migrations* (migrações de *schema*). Essa prática de engenharia de software permite que a criação das tabelas e as futuras alterações estruturais sejam versionadas como código. Na arquitetura proposta, as *migrations* são executadas de forma automatizada durante a inicialização do contêiner do banco de dados, garantindo que as alterações sejam aplicadas de maneira controlada e segura, preservando a consistência dos dados em qualquer ambiente de implantação.

5.5 MOTOR DE DETECÇÃO E EXPLICABILIDADE

5.5.1 Dataset e Preparação dos Dados

O conjunto de dados utilizado nesta pesquisa é o *Phishing Dataset* (ealvaradob/phishing-dataset) (ALVARADO, 2024), disponível na plataforma Hugging Face. Este repositório é caracterizado por sua natureza multimodal, consolidando informações de diversas origens consagradas na segurança da informação, como o *Enron Email Dataset*, *PhishTank* e *OpenPhish*.

Diferente de abordagens que se limitam a apenas um vetor de ataque, este trabalho utilizou a totalidade dos dados disponíveis no repositório para o ajuste fino do modelo classificador. O *dataset* é composto por três subconjuntos principais, totalizando aproximadamente 77.568 instâncias:

- **Textos (*texts*):** Amostras de corpos de e-mails e mensagens SMS (24.364 registros), focadas na semântica da engenharia social e manipulação psicológica.
- **URLs (*urls*):** Endereços web (20.735 registros) que permitem ao modelo identificar padrões maliciosos em domínios encurtados ou enganosos.

- **Web (*webs*):** Códigos-fonte HTML de páginas de *phishing* (32.469 registros), fornecendo informações sobre a estrutura técnica de sites fraudulentos.

Apesar do escopo deste trabalho ser apenas para classificação de dados textuais, para a etapa de ajuste fino do modelo **DistilBERT**, foi realizado um processo de unificação desses subconjuntos. Essa estratégia permitiu que o classificador desenvolvesse uma sensibilidade abrangente, sendo capaz de processar desde o conteúdo textual de uma mensagem até a estrutura bruta de um código HTML ou a sintaxe de uma URL suspeita.

A rotulagem mantém o padrão binário (*label* 0 para legítimo e 1 para *phishing*). Ao consolidar todas as modalidades de dados, a base de treinamento oferece uma representação robusta do ecossistema de ataques, garantindo que as explicações geradas posteriormente pelas etapas de SHAP e RAG tenham um fundamento técnico sólido sobre múltiplos indicadores de comprometimento.

Cabe ainda destacar que este trabalho utilizou dados textuais em língua inglesa, em razão da dificuldade de se encontrar conjuntos de dados dessa natureza em português disponíveis para uso livre e devidamente rotulados. Por esse motivo, optou-se por extrair as métricas de avaliação também a partir de dados em inglês, garantindo coerência entre o idioma do conjunto de treinamento e o idioma dos dados de teste e evitando resultados enviesados pela mudança de idioma. Ressalta-se, no entanto, que essa escolha decorre exclusivamente da disponibilidade de dados, e não de uma limitação da arquitetura proposta. O arcabouço desenvolvido é agnóstico em relação ao idioma. Sua aplicação a outras línguas, como o português, depende apenas do treinamento do classificador e da estruturação da base de conhecimento vetorial com documentos no idioma correspondente.

5.5.2 Módulo de Classificação e Interpretabilidade Local

5.5.2.1 Seleção do Modelo e Tokenização

Utilizou-se o modelo **DistilBERT** (*distilbert-base-uncased*) (SANH et al., 2020), uma versão compacta e eficiente do BERT que preserva a maioria das capacidades de compreensão semântica com menor custo computacional, sendo composto de 67 milhões de parâmetros

O processo de tokenização utiliza o algoritmo **WordPiece** (SCHUSTER; NAKAJIMA, 2012). Do ponto de vista técnico, este componente atua como um algoritmo estatístico de compressão e busca de padrões, segmentando o texto em subunidades (*subwords*).

5.5.2.2 Ajuste fino do classificador usando LoRA

Apesar do DistilBERT ser um modelo com uma quantidade relativamente grande de parâmetros, ainda é computacionalmente caro fazer o treinamento dele em computadores não muito robustos. Para contornar essa situação foi usado a técnica de **Low Rank**

Adaptation (LoRA) (HU et al., 2021), que reduziu drasticamente o número de parâmetros treináveis do modelo e, consequentemente, o tempo de treinamento do ajuste fino. Os metaparâmetros e métricas de treino usados foram:

Tabela 1 – Métricas do treinamento com LoRA

Métrica	**Valor**
Global Step	11 652
Loss de Treinamento	0.1594
Tempo Total de Treinamento (s)	1 525.36
Amostras por Segundo	122.22
Passos por Segundo	7.64
FLOPs Totais	1.26×10^{16}
Época Final	3.0

5.5.2.3 Extração de Relevância com SHAP

Para a interpretabilidade local, foi implementado o *KernelExplainer* da biblioteca **SHAP** (*SHapley Additive exPlanations*) (LUNDBERG; LEE, 2017). O sistema extrai os valores de contribuição marginal para a predição da classe positiva.

O pós-processamento dos dados SHAP envolve a filtragem dos *tokens* com maiores pesos positivos ($\phi > 0.05$). Estes *tokens* são convertidos em uma lista de palavras-chave que servirá como entrada para a consulta ao banco de dados vetorial.

5.5.3 Base de Conhecimento Vetorial (RAG)

A fundamentação técnica do sistema baseia-se na técnica de RAG, garantindo que as explicações não sofram de alucinações e estejam vinculadas a padrões reconhecidos de cibersegurança.

5.5.3.1 Ingestão e Estruturação de Documentos

Os manuais da **ENISA** (*Threat Landscape*) (European Union Agency for Cybersecurity (ENISA), 2025) e o framework **MITRE ATT&CK** (STROM et al., 2018) foram processados e carregados no banco de dados vetorial **ChromaDB** (TROYNIKOV et al., 2023). O processo de ingestão seguiu o seguinte pipeline:

- **Fragmentação (*Chunking*):** Divisão dos documentos em blocos de 1.000 caracteres com *overlap* de 200 caracteres para garantir a continuidade do contexto.

- **Vetorização (*Embedding*):** Utilizou-se o modelo *all-MiniLM-L6-v2* (WANG et al., 2020). Diferente do modelo generativo, este modelo de *embedding* é especializado na representação matemática de similaridade semântica em um espaço vetorial.

5.5.3.2 Mecanismo de Busca Semântica

A consulta ao **ChromaDB** é realizada através da similaridade de cosseno. O sistema utiliza os principais *tokens* identificados pelo SHAP para recuperar os dois fragmentos de texto tecnicamente mais relevantes, fornecendo a base teórica para a tática de engenharia social em curso (ex: *pretexting*, *urgency*, *authority*). Matematicamente, a similaridade de cosseno entre o vetor da consulta ($\mathbf{A}$) e o vetor do fragmento armazenado ($\mathbf{B}$) é calculada como o produto escalar normalizado pela magnitude dos vetores (SALTON; WONG; YANG, 1975):

$$\text{similaridade}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}} \tag{5.1}$$

Neste contexto, o valor resultante varia no intervalo [0, 1] para vetores de *embeddings* positivos, onde um valor próximo de 1 indica que o fragmento de conhecimento recuperado possui alta aderência semântica aos termos identificados como críticos pela análise de explicabilidade.

5.5.4 Módulo de Síntese de Explicação

O componente final do fluxo de análise é responsável por sintetizar a justificativa compreensível ao usuário. Durante a concepção deste módulo, um dos principais desafios práticos enfrentados foi o alto consumo de recursos e o tempo de inicialização exigido pelos modelos de Inteligência Artificial. Inicialmente, o componente de geração de texto utilizava o modelo local Qwen, mas, por exigir uma infraestrutura com GPU dedicada, essa abordagem se mostrou inviável para o ambiente de testes em CPU, frequentemente resultando em falhas por falta de memória, mesmo com a aplicação de técnicas de quantização em 4 bits (via biblioteca *BitsAndBytes*).

Para contornar esse desafio de hardware e garantir tempos de resposta rápidos, o processamento da geração de texto foi transferido para a nuvem. Dessa forma, a arquitetura final utiliza um modelo de LLM acessado via API da OpenAI, integrado ao sistema por meio da biblioteca LangChain (CHASE, 2022). Visando a flexibilidade e a evolução contínua da ferramenta, o nome do modelo utilizado não é fixado diretamente no código. Em vez disso, ele é parametrizado pela variável de ambiente `OPENAI_MODEL` (utilizando, por exemplo, o `gpt-5.4-mini`). Essa decisão de projeto permite que diferentes modelos sejam testados e substituídos no futuro apenas alterando o arquivo de configuração do contêiner.

Adicionalmente, a chamada à API do modelo de linguagem foi implementada como um recurso opcional, controlada pela variável `ENABLE_LLM`. Para garantir a resiliência do sistema em caso de instabilidade do serviço da OpenAI, foi aplicado o princípio de degradação graciosa. Se a API externa falhar, o Motor não rejeita a requisição. Em vez disso, ele retorna uma explicação estruturada de contingência baseada exclusivamente nos termos do SHAP e no contexto do RAG. Dessa forma, a classificação de ameaças continua funcional independentemente de falhas em integrações externas.

5.5.4.1 Engenharia de Prompt

A integração dos componentes é orquestrada por um *template* de prompt estruturado. Optou-se por manter esse *template* em língua inglesa, tendo em vista que os LLMs tendem a apresentar respostas de melhor qualidade quando instruídos nesse idioma (DEY et al., 2024). O *prompt* é construído dinamicamente concatenando o contexto recuperado do RAG, a importância dos termos do SHAP e o conteúdo do e-mail original. As regras de resposta instruem o modelo a evitar termos técnicos densos e a focar na relação entre a teoria (RAG) e a prática (e-mail analisado).

Como pode ser observado no Quadro 7, o *template* de *prompt* foi estruturado para orientar o modelo de linguagem na geração de explicações em linguagem natural, utilizando as atribuições de relevância extraídas via SHAP e o contexto recuperado pelo sistema RAG .

Quadro 7 – Template de prompt utilizado para a geração de explicações.

```
PROMPT_TEMPLATE = """

A transformer-based AI system was used to classify an email as phishing or not;
the result was: {classificacao}. Your task is to explain the justification for this
classification to the end user-the person who received the email.

IMPORTANT - GENERAL RULES:
1. Your focus should ONLY be explaining the decision of the transformer-based AI,
judge if the tokens with the highest SHAP values will help the end user (an human)
to understand the decision, if they don't DON'T bother to mention them in the final
explanation and try your best to justify the decision.
2. DO NOT introduce words, patterns, risks, or explanations that do NOT explicitly
appear in the SHAP data or the email.
3. Try to make sense with the words that have high scores and try to create a
narrative of why they have this score
4. When citing a token, use the exact form in which it appears in {top_features}
or in the text.
5. When describing each token, strictly respect its SHAP polarity:
   - POSITIVE values → contributed to appearing suspicious.
   - NEGATIVE values → contributed to appearing legitimate.
6. DO NOT use decimal numbers; use expressions such as "strong influence,"
"moderate influence," or "slight influence."

TONE CONTROL BASED ON CONFIDENCE:
7. If the total confidence value is lower than 0.75,
use conditional language such as "suggests," "indicates," or "may be related to."
8. If the confidence is equal to or higher than 0.75, use more direct language.
9. You should mention that the phishing classifier may commit mistakes and that is
the end-user responsability to make the final decision if they should trust or not
the e-mail.
INFLUENCE DATA (SHAP):
{top_features}

EMAIL CONTENT:
"{text}"

RETRIEVED CONTEXT:
{contexto}

TASK:
Produce a clear, simple explanation that is fully aligned with the data.
"""
```

5.5.4.2 Fluxo de Execução e Orquestração

A orquestração entre o banco de dados e o gerador de texto foi implementada utilizando a biblioteca **LangChain**. Esta ferramenta permite que a saída da consulta vetorial seja injetada diretamente na janela de contexto do LLM, automatizando o fluxo de inferência e garantindo que o sistema funcione de ponta a ponta sem intervenção manual.

5.5.5 API FastAPI e Gerenciamento do Ciclo de Vida

A arquitetura utiliza o *framework* FastAPI para expor o fluxo de análise ao ecossistema de microsserviços. Essa escolha é justificada pelo seu bom desempenho em Python e pelo suporte nativo a operações assíncronas, além da forte integração com a biblioteca *Pydantic*. Isso permite validar automaticamente os dados de entrada e saída por meio de tipagem estática, eliminando a necessidade de desenvolver rotinas manuais de validação no código.

Apesar da delegação do processamento de texto para a nuvem, o carregamento do classificador DistilBERT, dos adaptadores LoRA, do mecanismo do SHAP e a conexão com o banco ChromaDB ainda exigem um tempo computacional considerável. Para evitar lentidão na primeira análise solicitada pelo usuário, foi implementada uma rotina de carregamento prévio na inicialização do servidor, ativada pela variável `PRELOAD_ON_STARTUP`. Isso garante que as dependências analíticas sejam alocadas em memória antes que o serviço comece a receber tráfego. Como o servidor lida com requisições de forma concorrente, também foi necessário proteger a etapa de inicialização contra condições de corrida. A aplicação utiliza um mecanismo de bloqueio com dupla verificação, usando a primitiva `threading.Lock()`, para garantir que os modelos locais sejam carregados apenas uma vez, mesmo se múltiplas requisições chegarem ao servidor simultaneamente antes da conclusão do carregamento.

Para o monitoramento do ciclo de vida da aplicação, a API expõe duas rotas operacionais. A rota `GET /health` indica de forma simples que o servidor web está ativo. Já a rota `GET /ready` monitora o estado real do Motor, retornando o código HTTP 503 (Service Unavailable) enquanto os modelos estão sendo carregados e o código HTTP 200 (OK) somente após a inicialização com sucesso. O monitoramento do Docker Compose utiliza essa validação de estado para liberar o tráfego do Orquestrador apenas quando o Motor de Detecção está totalmente operacional.

5.6 IMPLANTAÇÃO E INFRAESTRUTURA

Para simplificar a configuração do ambiente e evitar conflitos de dependências, toda a infraestrutura do projeto é executada de forma isolada em contêineres Docker. O gerenciamento desse ecossistema é realizado pelo Docker Compose, que inicializa e interliga cinco serviços de uma só vez: o banco de dados (`postgres`), o Motor de Detecção em Python

(`ai-service`), o Orquestrador em Go (`api`), a interface web (`frontend`) e a documentação visual da API (`swagger-ui`).

Na prática, um dos maiores desafios ao integrar esses microsserviços é garantir a ordem correta de inicialização. O Orquestrador, por exemplo, não pode começar a receber requisições se o banco de dados não estiver pronto para gravar informações ou se o Motor ainda estiver alocando os modelos na memória. Para resolver essa questão, foram configuradas verificações de saúde diretamente no arquivo do Docker Compose. O banco PostgreSQL é monitorado pelo seu comando nativo `pg_isready`, enquanto o Motor é validado pela rota `/ready`. Assim, utilizando a regra `depends_on`, a API do Orquestrador permanece em espera e só inicia suas operações quando as dependências confirmam que estão totalmente estáveis.

Com foco na implantação em produção, a arquitetura foi desenhada para ser nativa em nuvem. Isso significa que nenhuma credencial ou endereço de servidor foi fixado no código. Todas as configurações são injetadas dinamicamente por meio de variáveis de ambiente no arquivo `.env`. Contudo, como servidores em nuvem com capacidade para executar modelos pesados de Inteligência Artificial possuem um custo elevado, a validação deste protótipo foi mantida restrita a um ambiente local.

6 EXPERIMENTOS E RESULTADOS

6.1 AVALIAÇÃO QUANTITATIVA

Para avaliar o desempenho do classificador proposto, foi construído um conjunto de dados sintético composto por 50 mensagens balanceadas, sendo 25 e-mails legítimos e 25 e-mails de *phishing*. A geração desses dados buscou simular cenários frequentemente observados em campanhas reais de engenharia social, incluindo falsas notificações bancárias, atualizações de credenciais corporativas, cobranças fraudulentas, falsas premiações e comunicações indevidas em nome de órgãos governamentais. A criação desse conjunto teve como objetivo disponibilizar um *dataset* independente, contendo exemplos representativos de mensagens legítimas e de *phishing* acompanhadas de justificativas textuais, possibilitando a avaliação tanto do desempenho do classificador quanto da qualidade das explicações geradas pelo sistema. Optou-se pela elaboração dos e-mails em língua inglesa devido ao fato de o modelo DistilBERT utilizado neste trabalho ter sido treinado e ajustado originalmente nesse idioma. Essa decisão permite uma avaliação mais consistente de sua capacidade de classificação e de geração de explicações, eliminando possíveis ruídos ou vieses decorrentes de traduções ou adaptações linguísticas.

A avaliação do desempenho do classificador baseia-se na classificação binária das mensagens, a partir das seguintes grandezas fundamentais:

- **Verdadeiro Positivo (VP):** Mensagens de *phishing* corretamente classificadas como maliciosas pelo modelo.
- **Verdadeiro Negativo (VN):** Mensagens legítimas corretamente classificadas como seguras.
- **Falso Positivo (FP):** Mensagens legítimas incorretamente classificadas como phishing.
- **Falso Negativo (FN):** Mensagens de *phishing* incorretamente classificadas como legítimas.

A partir dessas quatro grandezas, o comportamento e a eficiência do algoritmo são mensurados matematicamente segundo as Equações 6.1, 6.2, 6.3 e 6.4:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN} \tag{6.1}$$

$$\text{Precisão} = \frac{VP}{VP + FP} \tag{6.2}$$

$$\text{Recall} = \frac{VP}{VP + FN} \tag{6.3}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}} \tag{6.4}$$

A distribuição e a agregação dos experimentos realizados sobre as amostras legítimas e maliciosas são sintetizadas pela matriz de confusão consolidada, apresentada na Equação 6.5:

$$MC = \begin{bmatrix} 25 & 0 \\ 2 & 23 \end{bmatrix} \tag{6.5}$$

A leitura da matriz evidencia os seguintes resultados práticos do experimento:

- VN = 25 mensagens legítimas foram classificadas corretamente;
- FP = nenhuma mensagem legítima foi classificada incorretamente como *phishing*;
- FN = 2 mensagens de *phishing* foram classificadas incorretamente como legítimas;
- VP = 23 mensagens de *phishing* foram classificadas corretamente.

Os índices de desempenho obtidos a partir desse mapeamento foram consolidados na Tabela 2:

Tabela 2 – Desempenho do classificador de *phishing*

Métrica	**Valor**
Acurácia	96,00%
Precisão	100,00%
Recall	92,00%
F1-Score	95,83%

Observa-se que o modelo apresentou excelente desempenho na identificação de mensagens de *phishing* e legítimas. Nenhum e-mail legítimo foi classificado incorretamente como *phishing*, resultando em precisão de 100%. Além disso, o modelo identificou corretamente 23 das 25 mensagens de *phishing*, alcançando recall de 92%. O F1-score de 95,83% demonstra um excelente equilíbrio entre precisão e capacidade de detecção, enquanto a acurácia geral de 96% evidencia a robustez do classificador no conjunto de dados avaliado.

6.1.1 Avaliação das Justificativas Geradas

A avaliação do desempenho do classificador, apresentada na Seção anterior, demonstra a capacidade do modelo em distinguir mensagens legítimas de tentativas de *phishing*. Entretanto, um dos principais objetivos deste trabalho consiste em fornecer ao usuário

justificativas compreensíveis sobre os motivos que levaram o sistema a determinada classificação. Dessa forma, tornou-se necessário avaliar também a qualidade das explicações produzidas.

Diferentemente da classificação, a avaliação de explicações em linguagem natural constitui uma tarefa aberta, para a qual métricas tradicionais, como acurácia ou F1-score, não são adequadas. Em função disso, trabalhos recentes têm empregado a estratégia conhecida como *LLM-as-a-Judge*, na qual um modelo de linguagem de grande porte é utilizado para avaliar automaticamente a qualidade das respostas produzidas por outro modelo. Estudos como G-Eval Gu et al. (2025) demonstram que avaliações baseadas em modelos de linguagem apresentam elevada correlação com julgamentos humanos quando conduzidas por meio de rubricas bem definidas. De maneira semelhante, pesquisas posteriores consolidaram essa abordagem como uma alternativa prática para avaliação automática de tarefas abertas de geração de texto (ES et al., 2025; GU et al., 2025).

Neste trabalho foi utilizado o modelo **Sonnet 4.6**, disponibilizado pela Anthropic, como avaliador automático das justificativas produzidas pelo sistema.

Seguindo a abordagem de avaliação baseada em rubricas proposta por G-Eval, optou-se por não utilizar apenas uma nota geral para avaliar as justificativas produzidas pelo sistema. Em vez disso, cada explicação foi analisada segundo diferentes dimensões de qualidade, permitindo uma avaliação mais detalhada dos seus pontos fortes e limitações. Para o contexto deste trabalho, foram adotados quatro critérios: *Correctness*, *Completeness*, *Faithfulness* e *Clarity*, definidos a partir da adaptação de métricas amplamente empregadas na literatura de avaliação automática de modelos de linguagem e sistemas RAG (LIU et al., 2023; ES et al., 2025; GU et al., 2025).

A abordagem proposta por G-Eval baseia-se no conceito de *rubric-based evaluation* (avaliação baseada em rubricas). Nesse contexto, uma rubrica consiste em um conjunto de critérios previamente definidos que orientam o processo de avaliação de uma resposta. Em vez de solicitar ao modelo uma nota única e subjetiva, cada dimensão da resposta é analisada separadamente segundo critérios específicos. Essa estratégia reduz a subjetividade da avaliação, aumenta sua reprodutibilidade e permite identificar quais aspectos da resposta apresentam melhor ou pior desempenho. Liu et al. (2023) demonstram que avaliações conduzidas dessa forma apresentam maior correlação com julgamentos humanos quando comparadas a avaliações baseadas apenas em uma nota geral. Cada justificativa foi avaliada segundo quatro dimensões complementares:

- **Correctness**: avalia se a explicação identifica corretamente os fatores que justificam a classificação realizada pelo sistema. Esse critério corresponde ao conceito de *factual correctness*, amplamente utilizado na avaliação de respostas geradas por modelos de linguagem.

- **Completeness**: mede o quanto a justificativa cobre os principais elementos pre-

sentes na explicação de referência. Esse conceito é frequentemente empregado em avaliações baseadas em rubricas para verificar a cobertura das informações relevantes.

- **Faithfulness**: verifica se a justificativa permanece fiel às evidências presentes no e-mail, evitando informações inventadas ou conclusões não suportadas pelo conteúdo analisado. Esse critério é amplamente utilizado em trabalhos sobre sistemas RAG e avaliação de respostas fundamentadas, como proposto por Es et al. (2025).

- **Clarity**: avalia a facilidade de compreensão da justificativa por usuários finais, considerando organização, objetividade e legibilidade da explicação.

Além dessas quatro dimensões, o modelo avaliador atribuiu uma nota geral (*Overall*), variando entre 0 e 10, representando sua percepção global sobre a qualidade da justificativa.

Para os e-mails classificados como *phishing*, foi utilizada como referência uma justificativa (*ground truth*) construída manualmente durante a etapa de síntese do conjunto de dados. Essas justificativas descreviam explicitamente os principais indícios de engenharia social presentes em cada mensagem.

Já para os e-mails classificados como legítimos, o conjunto de dados original não possuía justificativas textuais. Assim, foi empregada uma etapa adicional de geração automática de *ground truth*, utilizando o próprio modelo GPT-5.5. Nessa etapa, o modelo recebeu apenas o conteúdo do e-mail e foi instruído a elaborar uma justificativa descrevendo exclusivamente os elementos que caracterizavam a mensagem como legítima, sem introduzir informações externas ao texto analisado. Essas justificativas foram então utilizadas como referência durante a etapa de avaliação.

A Tabela 3 e 4 apresentam os resultados médios obtidos para os dois conjuntos de dados.

Tabela 3 – Distribuição das avaliações das justificativas para os e-mails de phishing.

Critério	**Q1**	**Mediana**	**Q3**	**Média**
Correctness	7,00	7,50	8,00	7,04
Completeness	4,00	5,00	6,00	5,12
Faithfulness	7,00	9,00	9,00	7,69
Clarity	8,00	8,50	9,00	8,04
Overall	6,25	7,00	8,00	6,77

Tabela 4 – Quartis das avaliações das justificativas para e-mails legítimos.

Critério	**Q1**	**Mediana**	**Q3**	**Média**
Correctness	7,0	7,0	8,0	7,28
Completeness	5,0	5,0	6,0	5,30
Faithfulness	6,0	6,0	7,0	6,28
Clarity	7,0	8,0	8,0	7,48
Overall	6,0	6,5	7,0	6,47

A Tabela 4 apresenta as estatísticas descritivas das avaliações realizadas pelo LLM-as-a-Judge para os 25 e-mails legítimos. Os resultados indicam que o critério *Clarity* apresentou o melhor desempenho central, com mediana de 8,0 e primeiro quartil de 7,0, indicando que pelo menos metade das justificativas recebeu pontuação igual ou superior a 8,0 nesse aspecto. O critério *Correctness* também apresentou resultados elevados, com mediana de 7,0 e terceiro quartil de 8,0.

Por outro lado, *Completeness* apresentou o menor desempenho, com mediana de 5,0 e primeiro quartil também igual a 5,0. Esse resultado indica que uma parcela significativa das justificativas não contemplou de forma abrangente todos os elementos considerados relevantes pelo avaliador. O critério *Faithfulness* apresentou comportamento intermediário, com mediana de 6,0 e terceiro quartil de 7,0.

Por fim, a avaliação geral (*Overall*) apresentou mediana de 6,5, com 50% das observações concentradas entre 6,0 e 7,0. Esses resultados reforçam que as justificativas produzidas para os e-mails legítimos apresentam qualidade satisfatória em termos de clareza e correção, mas ainda possuem limitações principalmente quanto à abrangência das informações apresentadas.

De forma geral, as notas médias obtidas demonstram que a abordagem proposta produz justificativas coerentes, compreensíveis e majoritariamente fiéis às evidências utilizadas pelo classificador, complementando a elevada acurácia observada durante a avaliação quantitativa do modelo de detecção de *phishing*.

Para os e-mails classificados como *phishing*, a avaliação das justificativas por meio da abordagem *LLM-as-a-Judge* apresentou resultados positivos nas diferentes dimensões analisadas. A média obtida para o critério *Correctness* foi de 7,04, indicando que as justificativas produzidas pelo sistema, em geral, identificaram corretamente os principais fatores associados à classificação das mensagens como tentativas de phishing.

O critério *Faithfulness* apresentou uma média de 7,69, sendo o segundo maior resultado entre as dimensões avaliadas. Esse resultado é particularmente relevante para o sistema desenvolvido, uma vez que esse critério verifica se a justificativa permanece fundamentada nas evidências presentes no e-mail, evitando a inclusão de informações que não sejam sustentadas pelo conteúdo analisado. O resultado sugere que as explicações produzidas

pelo sistema apresentam, de maneira geral, boa correspondência com as características efetivamente presentes nas mensagens.

A maior média foi observada no critério *Clarity*, com 8,04. Esse resultado indica que as justificativas foram consideradas claras e compreensíveis pelo modelo avaliador. A clareza é especialmente importante no contexto da aplicação proposta, pois as explicações não são destinadas exclusivamente à análise técnica, mas também devem auxiliar o usuário na compreensão dos motivos que levaram o sistema a classificar uma mensagem como potencialmente maliciosa.

Por outro lado, o critério *Completeness* apresentou a menor média, com 5,12. Esse resultado indica uma limitação nas justificativas geradas: embora os principais indícios de phishing sejam frequentemente identificados, as explicações nem sempre contemplam todos os elementos relevantes presentes nas mensagens ou na justificativa de referência.

Considerando conjuntamente as quatro dimensões, a avaliação *Overall* apresentou média de 6,77. Esse resultado indica uma qualidade geral satisfatória das justificativas para os e-mails de phishing, embora ainda exista espaço para melhorias, principalmente no que diz respeito à completude das explicações. Dessa forma, os resultados sugerem que o sistema é capaz não apenas de identificar mensagens de phishing, mas também de fornecer justificativas que, em sua maioria, são corretas, fundamentadas e compreensíveis.

6.1.2 Avaliação de Latência e Desempenho Computacional

Além do desempenho preditivo do classificador, o requisito não funcional RNF03 estabelece um limite máximo de 120 segundos para a execução completa de uma análise. Esse tempo contempla todas as etapas do pipeline: classificação pelo DistilBERT, atribuição de relevância via SHAP, recuperação de contexto no ChromaDB (RAG) e geração da justificativa pelo LLM. Para avaliar a aderência do sistema a essa métrica, os tempos de execução de cada fase foram extraídos diretamente dos logs estruturados do Motor de Detecção e Explicabilidade. A coleta ocorreu ao longo de 48 requisições submetidas ao *endpoint* `/v1/phishing/analysis` em ambiente local, com a rotina de pré-carregamento de modelos (`PRELOAD_ON_STARTUP`) previamente concluída.

Durante a análise dos resultados, observou-se que o tamanho do conteúdo submetido influencia diretamente a latência total. Isso ocorre porque a etapa de atribuição de relevância via SHAP reavalia o modelo sobre partições do texto, o que torna seu custo computacional fortemente dependente do comprimento da entrada. Para detalhar esse impacto, os resultados foram divididos em três faixas de tamanho (curta, média e longa), contendo 16 requisições cada. A Tabela 5 apresenta o tempo total médio observado em cada faixa.

Tabela 5 – Latência total média por faixa de tamanho da entrada

Faixa	**Tempo médio (s)**	**Tempo mínimo (s)**	**Tempo máximo (s)**
Curta (450–714)	14,76	9,53	36,74
Média (732–1.232)	25,43	18,58	48,60
Longa (1.315–2.456)	63,05	39,45	109,80

Os resultados demonstram que a atribuição de relevância via SHAP é o principal gargalo de latência do pipeline, superando substancialmente o tempo da chamada ao LLM e da recuperação de contexto via RAG, que permaneceram abaixo de 1 segundo em todas as execuções. O caso mais longo efetivamente testado (2.456 caracteres, próximo ao teto de 2.500 estipulado no RF02) resultou em um tempo total de 109,80 segundos. Esse valor respeita o limite de 120 segundos do RNF03 com uma margem de segurança de aproximadamente 10%, comprovando a adequação do teto de caracteres adotado na arquitetura.

Adicionalmente, o tempo da primeira requisição da sessão de testes (578 caracteres processados em 36,74s) foi significativamente superior à média esperada para esse tamanho. Esse comportamento evidencia o efeito de *warm-up*: a primeira chamada ao *PartitionExplainer* do SHAP exige a inicialização de estruturas internas, um custo de processamento que não se repete nas execuções seguintes. Conclui-se, portanto, que a rotina de pré-carregamento adotada, embora aloque os modelos em memória, não elimina o custo de inicialização da primeira inferência do explicador. A mitigação desse atraso inicial fica como oportunidade de melhoria para trabalhos futuros.

6.1.3 Exemplo de Explicação com Recuperação de Conhecimento (RAG)

Para ilustrar de forma concreta o funcionamento do pipeline, esta seção apresenta um caso observado durante os experimentos em que as três etapas do sistema atuaram de forma particularmente satisfatória. Nesse caso, o sistema combinou três fontes de evidência:

1. A classificação realizada pelo modelo DistilBERT;
2. A explicação local produzida pelo SHAP, indicando os trechos mais influentes para a decisão;
3. Conhecimento externo recuperado por meio do mecanismo RAG.

O e-mail analisado continha uma suposta comunicação relacionada à Receita Federal, informando irregularidades no CPF do destinatário e solicitando acesso imediato a um link externo para regularização cadastral.

> Dear taxpayer,
>
> We inform you that your CPF registration has pending tax issues that require immediate action.
>
> Access the link below to verify and cancel the registration:
>
> http://www.receita-federal-regularizacao.com/cpf/cancelar
>
> Failure to act may result in additional charges.
>
> Sincerely

O modelo classificou a mensagem como **phishing** com confiança de **99,85%**.
As palavras e trechos com maior contribuição para a decisão, segundo o SHAP, foram:

- `immediately`
- `Access`
- `below`
- `taxpayer`
- `tax`
- `registration`
- `charges`
- `http://www.receita-federal-regularizacao.com/cpf/cancelar`

A explicação gerada pelo sistema foi:

> The AI classified this email as phishing because the strongest highlighted parts all pushed the decision toward phishing.
>
> The email tells the recipient to take action quickly, directs the user to access a link, references tax and identity-related consequences, and attempts to create urgency through warnings regarding CPF registration and possible charges.
>
> In plain terms, the model treated the email as suspicious because it combines an urgent request, tax-related warnings, and a link requesting user action, characteristics commonly associated with phishing campaigns.

Além dos indícios extraídos diretamente do modelo, o mecanismo RAG recuperou trechos de documentos de referência sobre campanhas de *phishing*. Entre os conteúdos recuperados destacam-se as seguintes definições:

> "Phishing is a form of social engineering where malicious actors lure victims (typically via email) to visit a malicious site or deceive them into providing login credentials."
>
> "Malicious actors pose as trustworthy organizations to lure a victim into interacting with a malicious hyperlink."

Observa-se que o conhecimento recuperado está diretamente relacionado ao comportamento identificado no e-mail analisado: utilização de uma entidade aparentemente confiável (Receita Federal), criação de senso de urgência e direcionamento do usuário para um link externo.

Os resultados demonstram o potencial da integração entre classificadores explicáveis e mecanismos de recuperação de conhecimento. Entretanto, devido às limitações de tempo e recursos computacionais disponíveis durante o desenvolvimento deste trabalho, não foi possível construir uma base vetorial suficientemente ampla para cobrir de forma abrangente todos os cenários analisados.

Consequentemente, apenas parte das explicações geradas durante os experimentos foi enriquecida com contexto relevante recuperado via RAG. Em uma implementação de produção, espera-se que uma base vetorial significativamente maior, contendo documentos especializados sobre *phishing*, engenharia social, segurança da informação e inteligência de ameaças, permita que todas as explicações sejam complementadas com evidências externas pertinentes, aumentando ainda mais a transparência e a confiabilidade do sistema.

7 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a avaliação do ExPhish, um sistema de apoio à decisão para detecção e explicação de tentativas de *phishing* em e-mails, com foco na mitigação do problema de "caixa-preta" de modelos de aprendizado profundo. A partir de um levantamento claro de requisitos funcionais e não funcionais, o ExPhish foi implementado estruturado em arquitetura de microsserviços, integrando um classificador DistilBERT ajustado via LoRA, a extração de relevância de termos com valores SHAP e a geração de explicações factuais em linguagem natural por meio de um arcabouço baseado em RAG (ChromaDB) e Grandes Modelos de Linguagem (LLMs). A solução conta ainda com um Orquestrador de Requisições responsável pela coordenação entre componentes, pela aplicação de mecanismos de resiliência (*timeout, retry* e *circuit breaker*) e pela persistência dos dados operacionais, além de um mecanismo de coleta e armazenamento de *feedback* dos usuários sobre as justificativas geradas, viabilizando o refinamento futuro do modelo de detecção.

Os resultados experimentais demonstraram, do ponto de vista quantitativo, que o ajuste fino do classificador foi suficiente para garantir alta confiabilidade na etapa de detecção. Em particular, o modelo ajustado alcançou uma acurácia de 96% e precisão de 100% no conjunto de dados avaliado, impactando diretamente na eliminação de falsos positivos. Do ponto de vista qualitativo, a avaliação das justificativas geradas, conduzida por meio da técnica de *LLM-as-a-Judge* segundo os critérios de correção, completude, fidelidade e clareza, indicou que a integração do mecanismo de RAG contribuiu para mitigar o risco de alucinações do LLM, fornecendo justificativas educacionais consistentes e contextualizadas com as boas práticas de segurança, ainda que o critério de completude tenha se destacado como o principal ponto de melhoria identificado.

A análise de engenharia e desempenho evidenciou que, apesar de a arquitetura de microsserviços com padrões de resiliência garantir alta disponibilidade em cenários de falha parcial, o pipeline sequencial introduz desafios de latência relevantes. Observou-se que o cálculo de atribuição de relevância do SHAP representa, isoladamente, a maior parcela do custo temporal total da solução, escalando de acordo com o tamanho do texto submetido. Adicionalmente, embora a terceirização da geração de texto para a API da OpenAI em nuvem tenha evitado sobrecargas de hardware local e mantido tempos de resposta na casa de milissegundos, essa abordagem introduz uma dependência estrita de conectividade, podendo gerar gargalos de rede em cenários produtivos com fluxos intensos de mensagens.

De forma geral, os resultados obtidos confirmam a viabilidade da construção de um ecossistema modular e funcional, capaz de aliar a alta capacidade preditiva do aprendizado profundo à interpretabilidade necessária para o usuário final no domínio da segurança da

informação. A modularização da arquitetura também se mostrou eficaz, permitindo o isolamento e a evolução independente dos componentes de inferência, inteligência explicativa e armazenamento vetorial de maneira transparente.

Como limitações deste trabalho, destacam-se a latência agregada no pipeline completo de geração de explicações e a dependência de infraestrutura proprietária em nuvem para a execução do modelo de linguagem. Adicionalmente, os experimentos e as avaliações concentraram-se em dados textuais no idioma inglês para maximizar o desempenho contextual da LLM, e os mecanismos planejados para a sanitização e o mascaramento prévio de informações pessoalmente identificáveis (PII) não foram integralmente consolidados nesta etapa. Por fim, embora o mecanismo de coleta de *feedback* tenha sido implementado e validado, sua utilização efetiva para o retreinamento do modelo de detecção não foi explorada neste trabalho.

Por fim, ressalta-se que ainda são necessários estudos adicionais para o aprofundamento do tema. Trabalhos futuros podem explorar a evolução da infraestrutura de implantação, migrando do ambiente local para plataformas de orquestração em nuvem para viabilizar o uso prático da ferramenta, podendo inclusive evoluir para um *plugin* instalado diretamente no servidor de e-mail, capaz de varrer e analisar mensagens automaticamente, sem exigir que o usuário copie e cole o conteúdo manualmente. Também pode-se explorar o enriquecimento da base vetorial do ChromaDB com um volume maior de documentos em outros idiomas, incluindo materiais em português, acompanhado do treinamento do classificador com dados nesse idioma. Além disso, há a possibilidade de usar técnicas mais enxutas no componente do RAG, como o hybrid search e o re-ranking, que podem melhorar signitificamente a qualidade dos textos recuperados. Como discutido na Seção 5.5.1, a arquitetura proposta é independente do idioma utilizado, de modo que esse trabalho futuro serviria para validar empiricamente essa característica, além de possibilitar a avaliação do sistema diante de táticas de ataque regionais. Essa mesma ampliação pode ser direcionada a contextos corporativos específicos, alimentando a base de conhecimento com cartilhas e políticas internas próprias da organização, de modo que a ferramenta atue como um suporte educativo alinhado às práticas de segurança de cada ambiente. Para mitigar as limitações de latência, pode-se também focar na evolução do Orquestrador de Requisições para um modelo assíncrono e orientado a eventos, dado que em um cenário ideal de alta escalabilidade, o tráfego não reteria conexões HTTP síncronas. No aspecto da segurança e privacidade, pode-se estudar a implementação de diferentes mecanismos de criptografia para os dados em repouso, avaliando o impacto no desempenho do sistema, uma vez que as análises armazenadas podem conter informações pessoais presentes nos e-mails analisados. Finalmente, aponta-se a necessidade de realizar um estudo qualitativo mais amplo com usuários finais, para validar de forma mais robusta a utilidade pedagógica das justificativas geradas pelo sistema.

REFERÊNCIAS

AFANE, K. et al. Next-generation phishing: How llm agents empower cyber attackers. In: **2024 IEEE International Conference on Big Data (BigData)**. [S.l.: s.n.], 2024. p. 2558–2567.

ALVARADO, E. **Phishing Dataset**. Hugging Face, 2024. Disponível em: https://huggingface.co/datasets/ealvaradob/phishing-dataset. Acesso em: 8 maio 2026.

CENSUSWIDE. **Phishing statistics: Only 3% of employees could confidently spot a scam**. 2026. Disponível em: https://dojo.tech/card-machine/phishing-statistics/. Acesso em: 22 jul. 2026.

CHASE, H. **LangChain**. 2022. Disponível em: https://github.com/langchain-ai/langchain. Acesso em: 10 maio 2026.

DEY, K. et al. **Better to Ask in English: Evaluation of Large Language Models on English, Low-resource and Cross-Lingual Settings**. 2024. Disponível em: https://arxiv.org/abs/2410.13153. Acesso em: 15 maio 2026.

DODGE, J. et al. **Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping**. 2020. Disponível em: https://arxiv.org/abs/2002.06305. Acesso em: 12 jul. 2026.

ES, S. et al. **Ragas: Automated Evaluation of Retrieval Augmented Generation**. 2025. Disponível em: https://arxiv.org/abs/2309.15217. Acesso em: 15 maio 2026.

European Union Agency for Cybersecurity (ENISA). **ENISA Threat Landscape 2025**. Luxembourg, 2025. Disponível em: https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025.

GAO, Y. et al. **Retrieval-Augmented Generation for Large Language Models: A Survey**. 2024. Disponível em: https://arxiv.org/abs/2312.10997. Acesso em: 29 jun. 2026.

GU, J. et al. **A Survey on LLM-as-a-Judge**. 2025. Disponível em: https://arxiv.org/abs/2411.15594. Acesso em: 15 maio 2026.

GUNNING, D. et al. Darpa's explainable artificial intelligence (xai) program. **AI Magazine**, AAAI Press, v. 40, n. 2, p. 44–58, 2019.

GUTTIKONDA, D. et al. Explainable ai: A retrieval-augmented generation based framework for model interpretability. In: INSTICC. **Proceedings of the 17th International Conference on Agents and Artificial Intelligence - Volume 3: ICAART**. [S.l.]: SciTePress, 2025. p. 948–955. ISBN 978-989-758-737-5.

HU, E. J. et al. **LoRA: Low-Rank Adaptation of Large Language Models**. 2021. Disponível em: https://arxiv.org/abs/2106.09685. Acesso em: 02 jun. 2026.

KALAI, A. T. et al. **Why Language Models Hallucinate**. 2025. Disponível em: https://arxiv.org/abs/2509.04664. Acesso em: 29 jun. 2026.

KRUCHTEN, P. B. The 4+ 1 view model of architecture. **IEEE software**, IEEE, v. 12, n. 6, p. 42–50, 1995.

LEWIS, J.; FOWLER, M. **Microservices**. 2014. Disponível em: https://www.martinfowler.com/articles/microservices.html. Acesso em: 10 maio 2026.

LEWIS, P. et al. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. 2021. Disponível em: https://arxiv.org/abs/2005.11401. Acesso em: 22 jun. 2026.

LIM, B. et al. **EXPLICATE: Enhancing Phishing Detection through Explainable AI and LLM-Powered Interpretability**. 2025. Disponível em: https://arxiv.org/abs/2503.20796. Acesso em: 10 maio 2026.

LIU, Y. et al. **G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment**. 2023. Disponível em: https://arxiv.org/abs/2303.16634. Acesso em: 04 maio 2026.

LUNDBERG, S.; LEE, S.-I. **A Unified Approach to Interpreting Model Predictions**. 2017. Disponível em: https://arxiv.org/abs/1705.07874. Acesso em: 02 jun. 2026.

MARTENS, D. et al. **Tell Me a Story! Narrative-Driven XAI with Large Language Models**. 2026. Disponível em: https://arxiv.org/abs/2309.17057. Acesso em: 04 maio 2026.

NEWMAN, S. **Building microservices: designing fine-grained systems**. [S.l.]: "O'Reilly Media, Inc.", 2021.

NIELSEN, J. **Response Times: The 3 Important Limits**. 1993. Disponível em: https://www.nngroup.com/articles/response-times-3-important-limits/. Acesso em: 7 jul. 2026.

SADALAGE, P. J.; FOWLER, M. **NoSQL Essencial: Um guia conciso para o mundo emergente da persistência poliglota**. [S.l.]: Novatec Editora, 2019.

SALTON, G.; WONG, A.; YANG, C.-S. A vector space model for automatic indexing. **Communications of the ACM**, ACM New York, NY, USA, v. 18, n. 11, p. 613–620, 1975.

SANH, V. et al. **DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter**. 2020. Disponível em: https://arxiv.org/abs/1910.01108. Acesso em: 02 jun. 2026.

SCHUSTER, M.; NAKAJIMA, K. Japanese and korean voice search. In: **2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**. [S.l.: s.n.], 2012. p. 5149–5152.

STROM, B. et al. **MITRE ATT&CK: Design and Philosophy**. 2018. The MITRE Corporation. Disponível em: https://attack.mitre.org/. Acesso em: 27 maio 2026.

TARAPIAH, S. et al. Evaluating the effectiveness of large language models (llms) versus machine learning (ml) in identifying and detecting phishing email attempts. **Algorithms**, v. 18, p. 599, 09 2025.

TROYNIKOV, A. et al. **Chroma: The open-source vector database**. 2023. Disponível em: https://www.trychroma.com. Acesso em: 02 jun. 2026.

VALENTE, M. T. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. [S.l.]: Editora: Independente, 2020.

VASWANI, A. et al. Attention is all you need. **CoRR**, abs/1706.03762, 2017. Disponível em: http://arxiv.org/abs/1706.03762. Acesso em: 29 jun. 2026.

VERIZON. **K12 SIX Contributes to Verizon 2025 Data Breach Investigations Report**. 2026. Disponível em: https://www.k12six.org/news/k12-six-contributes-to-verizon-2025-data-breach-investigations-report. Acesso em: 28 mar. 2026.

VOGELSANG, A.; FISCHBACH, J. **Using Large Language Models for Natural Language Processing Tasks in Requirements Engineering: A Systematic Guideline**. 2024. Disponível em: https://arxiv.org/abs/2402.13823. Acesso em: 15 maio 2026.

WANG, W. et al. **MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers**. 2020. Disponível em: https://arxiv.org/abs/2002.10957. Acesso em: 27 maio 2026.

WANG, Y. et al. Can you walk me through it? explainable sms phishing detection using llm-based agents. In: **Proceedings of the Twenty-First USENIX Conference on Usable Privacy and Security**. USA: USENIX Association, 2025. (SOUPS '25). ISBN 978-1-939133-51-9. Disponível em: https://www.usenix.org/system/files/soups2025-wang.pdf. Acesso em: 13 jun. 2026.

WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In: **Advances in Neural Information Processing Systems (NeurIPS 2022)**. [S.l.: s.n.], 2022. v. 35, p. 24824–24837.

ZENG, X.; ZHU, K. **Enhancing the Interpretability of SHAP Values Using Large Language Models**. 2026. Disponível em: https://arxiv.org/abs/2409.00079. Acesso em: 18 jun. 2026.

Zensec SOC employees. **Phishing statistics 2025 – 2026: The numbers you need to know**. 2026. Disponível em: https://zensec.co.uk/blog/2025-phishing-statistics-the-alarming-rise-in-attacks/. Acesso em: 10 jun. 2026.

ZHANG, J. et al. Benchmarking and evaluating large language models in phishing detection for small and midsize enterprises: A comprehensive analysis. **IEEE Access**, PP, p. 1–1, 01 2025.